

IMPLEMENTATION REPORT & USER MANUAL

Smart Grid & Asset Management

CSV asset import, validation and hierarchy explorer for substations, transformers, LV boards, switchboards and switchboard panels.

Version 1.2 · 21 September 2026

Source revision: **86edc0e** (branch main) plus uncommitted working-tree changes

Stack: Go (gin, GORM) · React + TypeScript · PostgreSQL 16 · Docker Compose

Contents

Contents

1. Introduction

- 1.1 Purpose of this document
- 1.2 What the system does
- 1.3 Scope and deliberate non-goals
- 1.4 Technology summary

2. Functional requirements

- 2.1 Requirements from the brief
- 2.2 Requirements added by the owner
- 2.3 Non-functional requirements
- 2.4 Traceability matrix
- 2.5 Use cases

3. System architecture

- 3.1 Deployment view
- 3.2 Layering
- 3.3 Request pipeline and access control
- 3.4 Concurrency: where it is used and where it is not
- 3.5 Configuration

4. Data design

- 4.1 Entity-relationship diagram
- 4.2 Reference data
- 4.3 Design decisions

5. Class diagrams

- 5.1 Domain model
- 5.2 Backend classes and their collaborators

6. Sequence diagrams

- 6.1 Check a CSV (preview): nothing is written
- 6.2 Confirm an import: revalidate, then commit in one transaction
- 6.3 Search, then locate the asset in the hierarchy
- 6.4 Browse the hierarchy: roots, then children with counts beneath
- 6.5 Delete one asset
- 6.6 API-key check on every request

7. Validation algorithm and import transaction

- 7.1 Rules and where each is enforced
- 7.2 Worked example: the supplied sample file
- 7.3 Import outcome: check → review → confirm (FR-1.11)
- 7.4 Flexible input handling

8. API reference

- 8.1 Endpoints
- 8.2 Status codes and error body
- 8.3 Response shapes worth knowing

- 8.4 Trying it with curl

9. Frontend implementation

- 9.1 Routes and layouts
- 9.2 The import flow as a state machine
- 9.3 Server state, caching and generated types
- 9.4 Design tokens, themes and layout
- 9.5 Contents by type (FR-3.4)
- 9.6 Per-panel states and accessibility
- 9.7 Asset-type icons and the footer

10. Testing and verification

- 10.1 Strategy
- 10.2 Results measured while preparing this document

11. Build and deployment (local)

- 11.1 Running the whole stack
- 11.2 Developer commands
- 11.3 Everyday and troubleshooting commands

12. Security and operations

13. User manual

- 13.1 Before you start
- 13.2 The screen at a glance
- 13.3 Importing assets from a CSV file
- 13.4 The Overview page
- 13.5 Browsing the hierarchy
- 13.6 The All assets list
- 13.7 Deleting an asset
- 13.8 Dark mode, phones and states
- 13.9 Keyboard reference
- 13.10 Troubleshooting
- 13.11 For administrators
- 13.12 Frequently asked questions

14. Known limitations and gaps

- 14.1 Limitations
- 14.2 Not done, and what would come next

15. AI usage journal

- 15.1 Tools, models and effort
- 15.2 How the work was organised
- 15.3 Session timeline
- 15.4 Suggestions rejected, corrected or revised
- 15.5 Known gaps and next steps

Appendix A. CSV template and sample data

Appendix B. Glossary

1. Introduction

1.1 Purpose of this document

This document describes how the Smart Grid & Asset Management system was implemented and how to use it. It has two audiences: **engineers** (chapters 2–12: requirements, architecture, data model, class and sequence diagrams, validation algorithm, API, testing, deployment) and **operators** (chapter 13: a step-by-step user manual with screenshots). Every screenshot was taken from the running application, and every diagram was checked against the source code at the revision on the cover.

1.2 What the system does

Utilities keep equipment in a strict hierarchy: a **substation** contains transformers, LV boards and switchboards; a **switchboard** contains switchboard panels. The system lets a user:

- **Import** a CSV of assets through a two-step *check* → *review* → *confirm* workflow. The backend validates every row (fields, uniqueness, parent existence, permitted parent types, cycles) and stores the valid rows in one transaction.
- **See the outcome:** total, imported and rejected rows, each rejection with its original CSV row, asset ID and a plain-language reason, and an explicit statement of whether data was committed.
- **Explore** the stored hierarchy as an expandable tree with a details panel, search, a filterable/sortable list of all assets, and a fleet overview.

1.3 Scope and deliberate non-goals

In scope	CSV import with validation, import audit trail, read-only explorer, search, list/stats endpoints, deleting one childless asset, Swagger/OpenAPI, local Docker Compose build and run (no CI/CD pipeline).
Access model	One shared API key (<code>x-api-key</code>) guards every <code>/api</code> route. The frontend never holds the key: nginx (Docker) or the Vite dev proxy adds it to each proxied request.
Out of scope	User accounts, roles/RBAC, login screen, editing an asset, create-one, upsert on import, delete-all. These are excluded on purpose (see <code>CLAUDE.md</code> non-negotiables) and are not omissions.

1.4 Technology summary

Layer	Choice	Why
Backend	Go, <code>gin</code> (HTTP), <code>gorm</code> + <code>pgx</code> (Postgres); raw SQL only for recursive CTEs	Strong concurrency primitives for validation; small static binary; GORM's builder keeps repositories declarative.
Database	PostgreSQL 16, migrations in <code>db/migrations</code> applied by a one-shot <code>migrate</code> service	Recursive CTEs for subtree/ancestor queries; <code>pg_trgm</code> indexes for search; real FKs and CHECKs as a last guard.
Frontend	React + TypeScript (Vite), React Router, TanStack Query, Tailwind tokens, Radix Dialog	Server state lives only in TanStack Query; the only client state is the expanded-tree set, the import flow and the mobile drawer.
API contract	swag annotations → Swagger 2 → OpenAPI 3 → generated <code>schema.ts</code>	Frontend types cannot drift from the API; <code>swagger_test.go</code> fails if a route and the spec disagree.
Delivery	Docker Compose: <code>db</code> → <code>migrate</code> → <code>backend</code> → <code>frontend</code> (nginx)	One command from a clean machine.

2. Functional requirements

The functional requirements come from section 4 of the project brief (4.1 CSV upload and validation, 4.2 Results, 4.3 Asset explorer). They are reproduced below in a numbered form so that every later chapter, and the traceability matrix in 2.4, can point at them by ID. Requirements that the owner added afterwards are listed separately in 2.2.

2.1 Requirements from the brief

4.1 — CSV upload and validation

Provide a frontend workflow for selecting and uploading a CSV file. The backend must parse and validate the file before persistence.

ID	Requirement
FR-1.1	A frontend workflow for selecting and uploading a CSV file.
FR-1.2	The backend parses and validates the file before persistence.
FR-1.3	<code>asset_id</code> , <code>asset_type</code> and <code>asset_name</code> are required.
FR-1.4	Asset IDs must be unique.
FR-1.5	Asset type and operational status must be supported values.
FR-1.6	Dates and numeric fields must be valid; <code>rating_kva</code> cannot be negative.
FR-1.7	A substation cannot have a parent; every other asset requires one.
FR-1.8	The parent must already exist in PostgreSQL or be present somewhere in the uploaded file.
FR-1.9	An asset cannot parent itself and the hierarchy cannot contain cycles.
FR-1.10	Every parent-child relationship must conform to the permitted hierarchy and ultimately resolve to a substation root.
FR-1.11	Choose either all-or-nothing or partial import; document the choice and make the outcome transactionally predictable.

4.2 — Results

ID	Requirement
FR-2.1	After processing, show total rows, imported rows and rejected rows.
FR-2.2	For every rejection, report the original CSV row, the asset ID where available, and a meaningful validation message.
FR-2.3	Make it clear whether data was committed.

4.3 — Asset explorer

ID	Requirement
FR-3.1	Display the stored hierarchy as an expandable tree.
FR-3.2	Provide a selected-asset details area.
FR-3.3	Selecting a substation shows its details and its immediate children grouped by type.
FR-3.4	Selecting a substation shows counts for transformers, LV boards, switchboards and switchboard panels beneath it.
FR-3.5	Provide appropriate loading, empty and error states.

Traceability note

The repository's architecture notes also refer to a brief section 4.4 “stable route” requirement (direct navigation and refresh must resolve the same asset). Only sections 4.1–4.3 were available as source text for this document, so that requirement is recorded as **FR-4.1** from the repository's description rather than from the brief itself.

ID	Requirement (as recorded in docs/ARCHITECTURE.md)
FR-4.1	Each asset has a stable URL (<code>/assets/:assetId</code>); direct navigation and browser refresh resolve the same asset.

2.2 Requirements added by the owner

At the owner's request (2026-09-20, recorded in `docs/ASSUMPTIONS.md`) the API grew beyond import-only. These are additional to the brief.

ID	Requirement
FR-5.1	Preview an import (validate, store nothing) and confirm it as a separate step.
FR-5.2	Download a CSV template and a CSV report of rejected rows.
FR-5.3	Import history: past imports, newest first, with who/what/when and their rejected rows.
FR-5.4	Search assets by ID or name (partial, case-insensitive) and open a result in the tree.
FR-5.5	List all assets, filter by text/type/status, sort by any column server-side, paged.
FR-5.6	Overview: totals by type, top-level assets, operational-status split.
FR-5.7	Delete one asset that has no children (409 otherwise). There is no delete-all.
FR-5.8	Every <code>/api</code> endpoint requires the shared API key; <code>/healthz</code> and the Swagger docs are open.
FR-5.9	Light and dark themes; usable on a phone (tree becomes a drawer).

2.3 Non-functional requirements

Area	Requirement and how it is met
Server-side truth	The backend re-validates everything the frontend checks; client validation is UX only. Confirm never trusts the preview.
Atomicity	Accepted rows and the audit rows are written in one DB transaction; a failure leaves nothing behind.
Security	Constant-time API-key comparison, header only; fails closed (403) when no key is configured; security headers; CORS allow-list; rate limit 100 requests / 10 s per client and path; body-size cap (default 10 MiB); log values quoted against log forging.
Performance	Pass 1 runs on a chunked worker pool; independent reads are overlapped (<code>errgroup</code>); subtree/ancestor queries are single recursive CTEs; <code>pg_trgm</code> indexes back search.
Observability	An 8-hex trace ID per request (<code>X-Trace-Id</code>) appears in every log line and error body; every rejection is logged (capped at 50 lines per import).
Accessibility	ARIA tree roles kept in sync with state, keyboard-operable tree and search combobox, <code>:focus-visible</code> never suppressed, WCAG AA (4.5:1) text contrast in both themes.
Maintainability	Strict layering (router → handler → service → repository), generated API types, colour/radius tokens defined once, spacing-scale test.
Portability	<code>docker compose up --build</code> from a clean checkout; nothing installed on the host except Docker.

2.4 Traceability matrix

Status was determined by reading the implementation and by exercising it in the running application (evidence column). Met = implemented and observed working; Partial = implemented in part. Nothing is Partial in this issue (v1.2): FR-3.4 was Partial in the first issue and was completed afterwards.

ID	Status	Where it is implemented	Evidence
FR-1.1	Met	<code>ImportPanel</code> , <code>useImportFlow</code> (dropzone, Check file)	Fig. 13.2–13.5
FR-1.2	Met	<code>base/csvx</code> (structure), <code>ImportService.validate</code> (rows); preview writes nothing	Preview response, Fig. 13.5
FR-1.3	Met	Pass 1 required-field check (also <code>operational_status</code> , which is <code>NOT NULL</code>)	Rows 182 (no id), 183 (no name)
FR-1.4	Met	Pass 1 duplicate-id (every occurrence rejected); pass 2 rule 0 (already stored); PK as last guard	Re-import rejects all stored ids
FR-1.5	Met	Lookup snapshot (<code>asset_types</code> , <code>operational_statuses</code>), case-insensitive match	Rows 178 (GENERATOR), 181 (BROKEN)
FR-1.6	Met	Pass 1 number/date fields; DB <code>CHECK (rating_kva >= 0)</code>	Rows 179 (bad date), 180 (–500 kVA)
FR-1.7	Met	Pass 2 rule 1 (root types come from the lookup table, not code)	Rows 184 (substation with parent), 185 (transformer without)
FR-1.8	Met	Pass 2 rule 3 over the union of DB ids and surviving file ids, so a child may precede its parent	Rows 91, 92; unit tests

ID	Status	Where it is implemented	Evidence
FR-1.9	Met	Pass 2 rule 2 (self-parent), rule 5 (3-colour cycle walk, O(n))	Rows 93–97 (two cycles); unit test “asset is its own parent”
FR-1.10	Met	Pass 2 rule 4 (<code>asset_type_parent_rules</code>) and rule 6 (cascade: “ancestor X was rejected”)	Rows 98, 99
FR-1.11	Met	Partial import behind a preview: valid rows commit in one transaction; fingerprint → 412; PK/FK collision → 409 (chapter 7)	Fig. 6.2a–b; 205 of 222 stored
FR-2.1	Met	<code>ImportPreview</code> , <code>ImportResult</code> summary tiles	Fig. 13.5, 13.6
FR-2.2	Met	<code>RejectionTable</code> : original file line (header = 1), asset id (blank when absent), reason listing every problem	Fig. 13.5–13.7
FR-2.3	Met	Banner “Data committed: yes — 205 rows stored, 17 rejected rows were not saved”; <code>committed</code> flag; “Nothing has been saved yet” on the preview	Fig. 13.5, 13.6
FR-3.1	Met	<code>AssetTree</code> / <code>AssetTreeNode</code> : roots first, children fetched on first expand	Fig. 13.11
FR-3.2	Met	<code>AssetDetailsPanel</code> (fields, parent link, children table)	Fig. 13.9
FR-3.3	Met	Details, per-type cards (with how many are direct) and a sortable immediate-children table (sorted by type by default, so grouped by type)	Fig. 13.9
FR-3.4	Met	Backend: GET <code>/children</code> returns <code>descendant_counts</code> (every asset beneath the node, at every depth, by type). Frontend: the Contents by type, at every level cards in <code>AssetDetailsPanel</code> show one card per type that can occur beneath the asset (the total at every depth, plus how many are direct), including 0; the immediate-children table below them lists the direct children. The types come from the parent rules (<code>descendantTypes</code> in <code>lib.ts</code>), not from code.	Fig. 13.9: Aurora (<code>SS-001</code>) shows 2 transformers, 1 LV board, 2 switchboards, 10 switchboard panels (0 direct, 10 deeper). 8 unit and component tests.
FR-3.5	Met	Each panel has its own skeleton, empty state and error state with Try again (<code>components/ui</code>)	Fig. 13.1, 13.17, 13.18
FR-4.1	Met	Route <code>/assets/:assetId</code> ; nginx <code>try_files ... /index.html</code>	Fig. 13.9, 13.18 (unknown id → not-found panel)
FR-5.1–5.9	Met	See chapters 8, 10 and 13	Fig. 13.5–13.19

2.5 Use cases

The single actor is anyone who can reach the UI (and so holds the shared key by proxy). “Confirm import” includes “Check CSV file”: a commit is only offered after a check. Reviewing rejected rows extends a check.

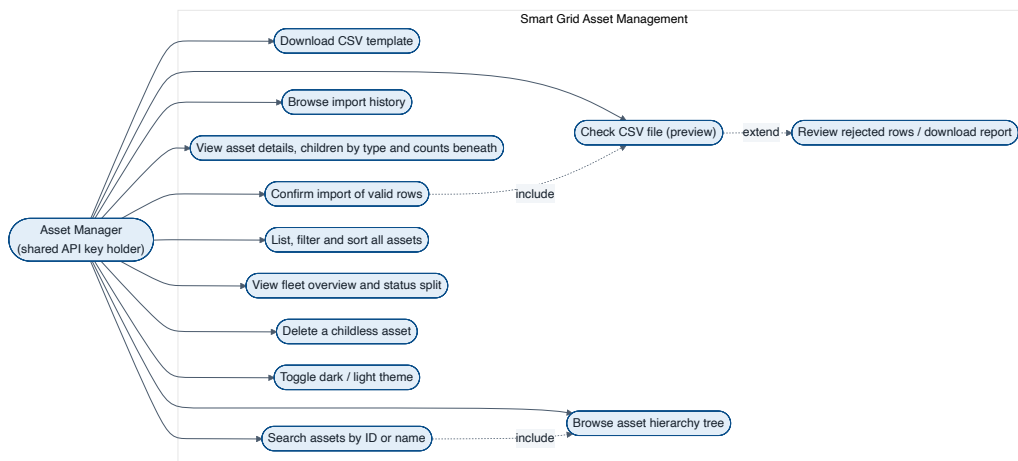


Figure 2.1 — Use-case diagram

3. System architecture

3.1 Deployment view

Four Compose services start in dependency order. `db` must be healthy before `migrate` runs; `migrate` must exit successfully before `backend` starts; `backend` must be healthy before `frontend` starts. The browser only ever talks to `nginx`. `nginx` serves the SPA and reverse-proxies `/api/*`, `/healthz` and the open Swagger pages (`/swagger/*`, `/docs`) to the backend. It adds the API key to `/api/*` only, so the key never reaches the browser or the JavaScript bundle, and the footer's **API docs** link works on the app's own port. There is no CI/CD pipeline: the stack is built and run locally with `docker compose up --build` (chapter 11).

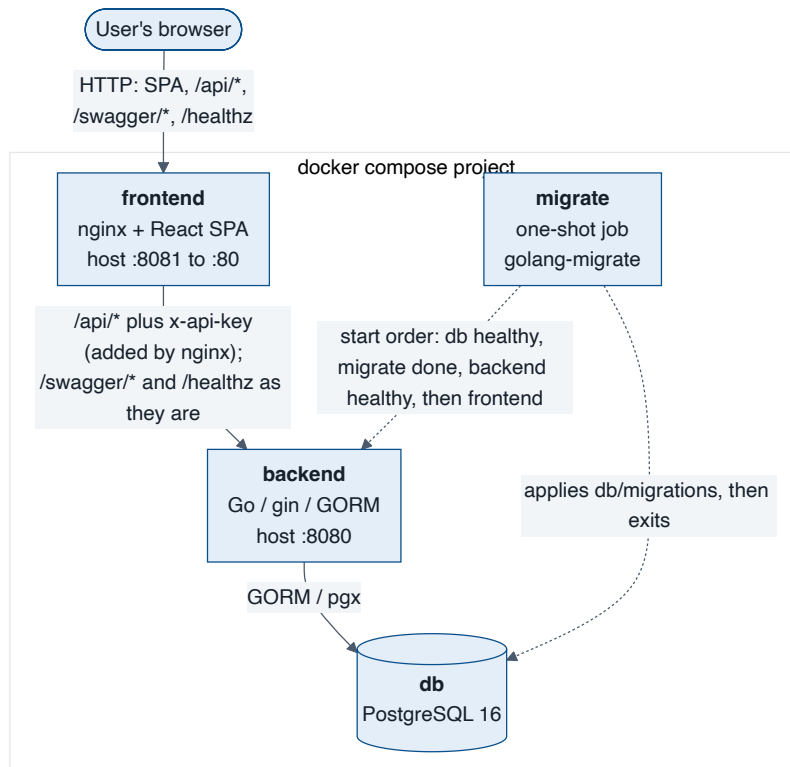


Figure 3.1 — Deployment diagram (docker compose)

3.2 Layering

Dependencies point one way. Handlers bind and write JSON only; services own the rules; repositories own SQL. `base/` is generic and imports no application package; `defined/` is a leaf every layer may import; `dto` never imports `model`. `handler/errors.go` is the only place that maps errors to HTTP statuses.

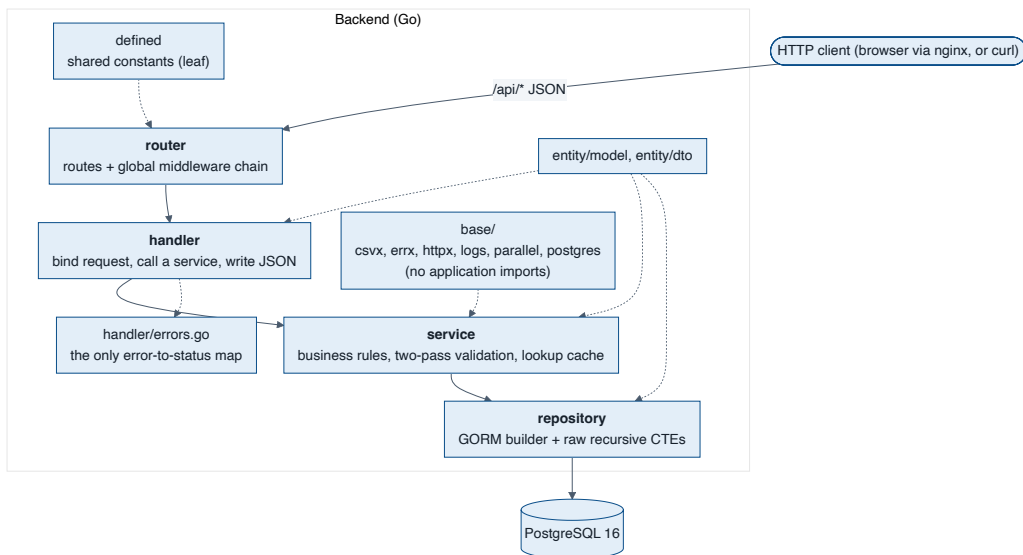


Figure 3.2 — Backend layering

Package	Responsibility
router	Every route and the global middleware chain in one file, so the endpoint set is readable at a glance.
middleware	Trace ID, access log, panic recovery, security headers, CORS, rate limit, size limit, API key.
handler	Bind request, call a service, write JSON. Import handler wraps each import in a 2-minute timeout.
service	<code>ImportService</code> (two-pass validation, commit), <code>AssetService</code> , <code>LookupCache</code> / <code>LookupSnapshot</code> , <code>HealthService</code> .
repository	GORM queries; raw SQL only for recursive CTEs (subtree counts, descendant counts, ancestors); <code>InTx</code> and write-error mapping.
entity/model, entity/dto	GORM-tagged models with <code>TableName()</code> and model→dto helpers; snake_case wire types, request types carry <code>Validate()</code> .
base/*	<code>csvx</code> (structural CSV upload), <code>errx</code> (sentinels), <code>httpx</code> , <code>logs</code> (trace-aware slog), <code>parallel</code> (chunked worker pool), <code>postgres</code> .
defined	Shared constants: search/list sizes, upload limits, header names, date formats, timeouts.

3.3 Request pipeline and access control

Order matters: `TraceID` first so everything downstream can log it; `AccessLog` wraps `Recover` so a recovered panic is logged as the 500 it became. Only the `/api` group passes through `RequireAPIKey`; `/healthz`, `/docs` and `/swagger/*` are open.

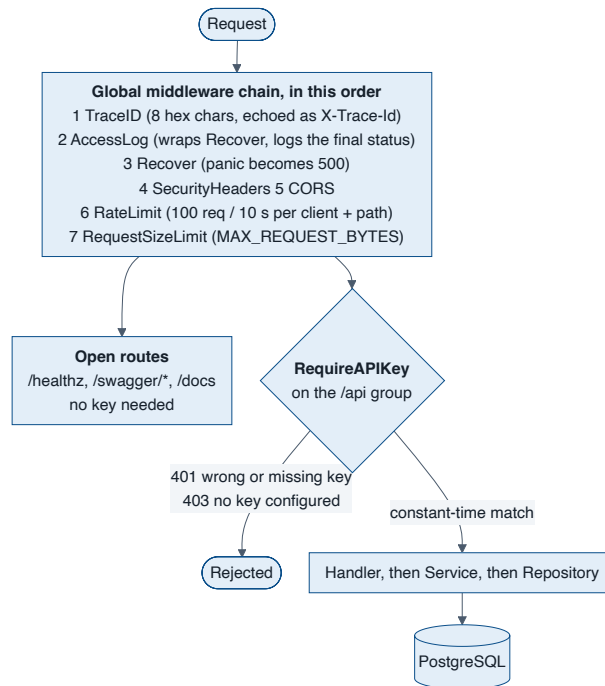


Figure 3.3 — Middleware chain and route groups

3.4 Concurrency: where it is used and where it is not

Where	How	Why
Import pre-validation reads	<code>errgroup</code> : lookup-table refresh runs alongside “which of these ids already exist”	Independent I/O, overlapped.
Pass 1	Chunked worker pool (<code>base/parallel</code> , 512 rows/chunk, at most GOMAXPROCS). Workers write disjoint indices of a pre-sized slice, so order is deterministic with no lock. Panics become errors; request cancellation is honoured.	CPU-bound; rows are independent.
GET <code>/children</code>	<code>errgroup</code> : parent exists children with counts descendant counts.	Three independent reads.
Deliberately not used	Pass 2, the transactional insert, subtree counts, ancestors, search.	Pass 2 shares graph state and is a few map lookups per row; a transaction is one connection; one recursive CTE beats N per-child queries.

3.5 Configuration

Variable	Meaning
<code>POSTGRES_USER/PASSWORD/DB/PORT</code>	Bootstraps the <code>db</code> container.
<code>DB_HOST/PORT/USER/PASSWORD/NAME/SSLMODE</code>	Backend and <code>migrate</code> connection, kept as discrete fields (no URL-escaping of passwords).
<code>BACKEND_PORT</code> , <code>FRONTEND_PORT</code>	Backend listen port (also published) and the host port of the web UI (default 8081).
<code>API_KEY</code>	Shared secret for every <code>/api</code> route. Generate with <code>openssl rand -hex 32</code> . If unset every <code>/api</code> call gets 403.
<code>CORS_ALLOWED_ORIGINS</code>	Comma-separated allow-list; empty means any origin (no cookies or credentials exist).
<code>MAX_REQUEST_BYTES</code>	Largest request body, hence largest CSV upload. Default 10 MiB.

4. Data design

4.1 Entity-relationship diagram

Asset types, operational statuses and the permitted child→parent pairs are **lookup tables**, not native ENUMs or Go constants. Adding a type, status or pairing is a plain `INSERT`, with no migration and no redeploy. Each lookup table uses its code as a natural primary key, so the API and CSV keep using the same strings.

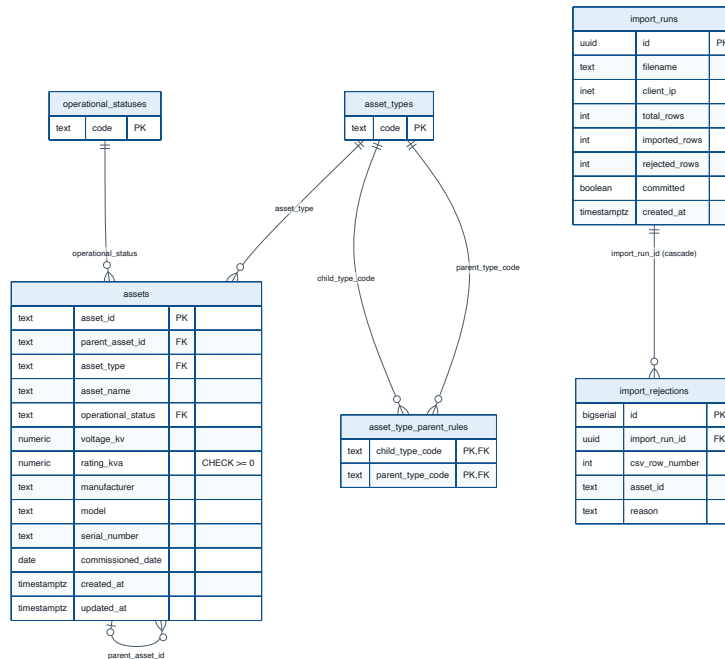


Figure 4.1 — ER diagram of the seven tables

4.2 Reference data

Table	Seed rows
<code>asset_types</code>	SUBSTATION, TRANSFORMER, LV_BOARD, SWITCHBOARD, SWITCHBOARD_PANEL
<code>operational_statuses</code>	IN_SERVICE, MAINTENANCE, OUT_OF_SERVICE
<code>asset_type_parent_rules</code>	TRANSFORMER→SUBSTATION, LV_BOARD→SUBSTATION, SWITCHBOARD→SUBSTATION, SWITCHBOARD_PANEL→SWITCHBOARD. A type with no row as a child (SUBSTATION) is a root type and must have no parent.

4.3 Design decisions

- **Every table has** `created_at` / `updated_at`, the latter maintained by one shared `set_updated_at()` trigger function attached per table (Postgres has no `ON UPDATE CURRENT_TIMESTAMP`).
- **Indexes:** `idx_assets_parent`, `idx_assets_type`, and GIN trigram indexes on `asset_name` and `asset_id` (`pg_trgm`) so substring search stays fast.
- **Constraints as a last guard:** `rating_kva >= 0` CHECK; self-referencing FK on `parent_asset_id`; FKs from `assets` to the lookup tables. A primary-key or FK violation during commit aborts the transaction and is answered with 409.
- **Hierarchy rules are enforced in the application layer**, not by a trigger (parent type pairing, cycles, cascade). Only CSV import and “delete one childless asset” write to `assets`, and both go through the service. A raw SQL `INSERT` could bypass the checks; nothing in the system does that.
- `import_runs.client_ip` (`INET NOT NULL`) stands in for a user id. It is an audit breadcrumb, not a security control (the backend port is also published, so `X-Forwarded-For` can be forged by a direct caller).
- `import_rejections` stores only line number, asset id and reason, not the rejected cell values.

5. Class diagrams

5.1 Domain model

The Go models mirror the tables (GORM tags, `TableName()`) and carry the projections onto API shapes (`Asset.ToSummary`, `ToDetail`, `ImportRun.ToResponse`). `AssetType` and `OperationalStatus` are plain strings: the lookup tables are the single source of truth, so there are no Go constants for the permitted values. * marks a nullable field.

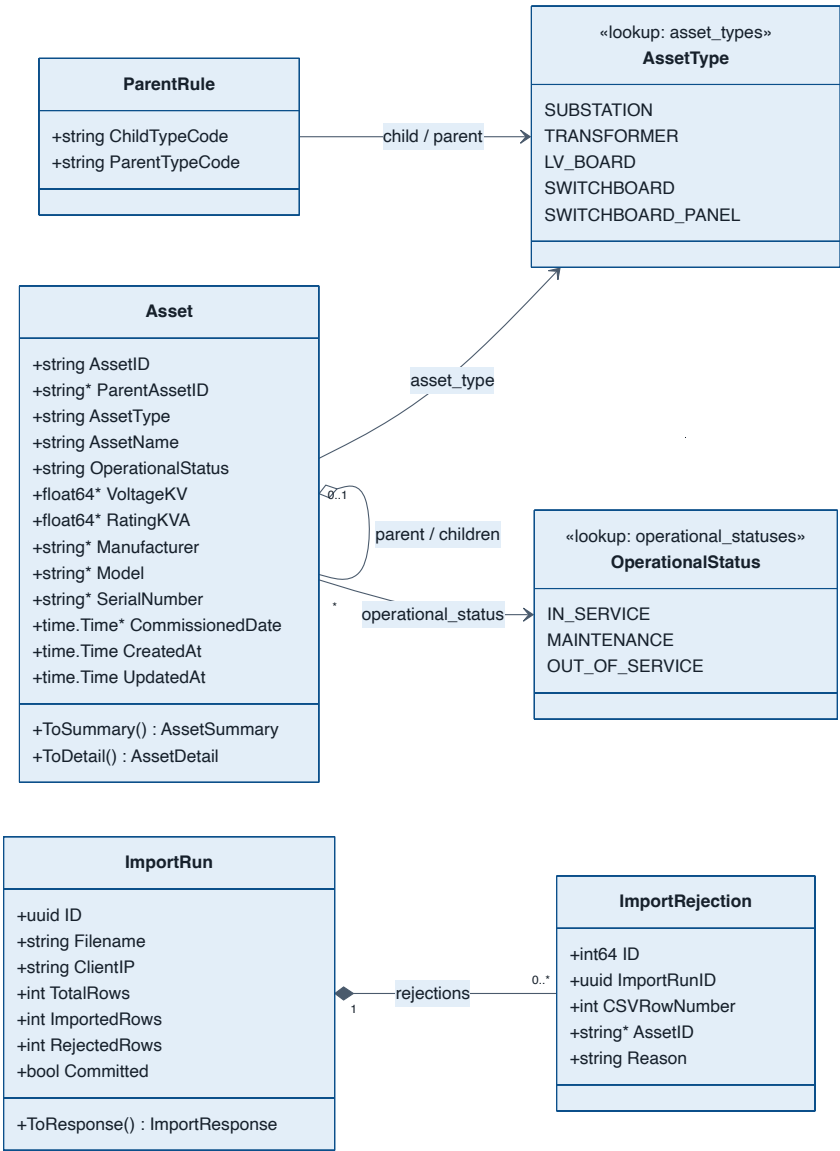


Figure 5.1 — Domain classes

5.2 Backend classes and their collaborators

Handlers depend on **small interfaces** (`importService`, `assetService`, `snapshotSource`) declared beside them, so they are tested with fakes.

`LookupCache` holds an immutable `LookupSnapshot` behind an atomic pointer: it is loaded at startup and refreshed at the start of every import, and one snapshot is used for a whole run so validation is internally consistent.

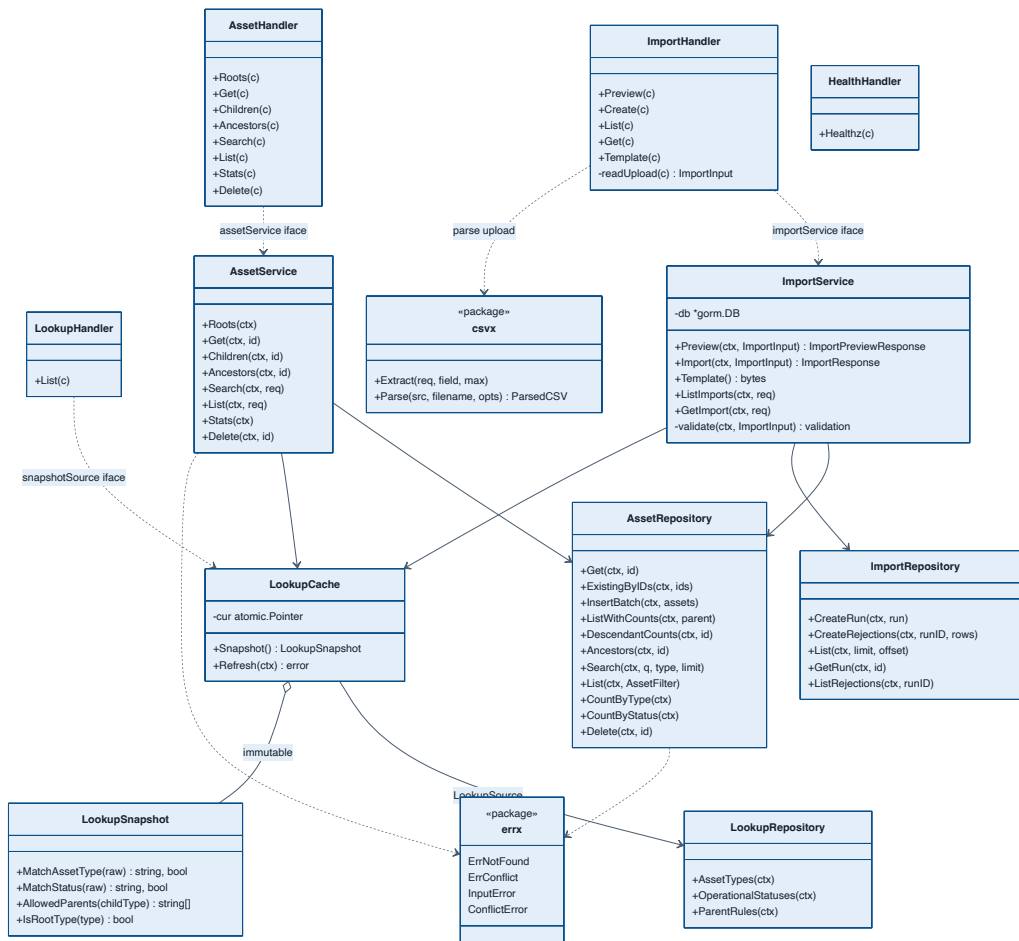


Figure 5.2 — Backend class diagram (handler → service → repository)

6. Sequence diagrams

The six interactions that carry the design. Each was traced through the handler, service and repository code, not drawn from the intent. The two longest (6.1 and 6.2) are drawn in two halves so the text stays legible; the notes at the join say where each half continues.

6.1 Check a CSV (preview): nothing is written

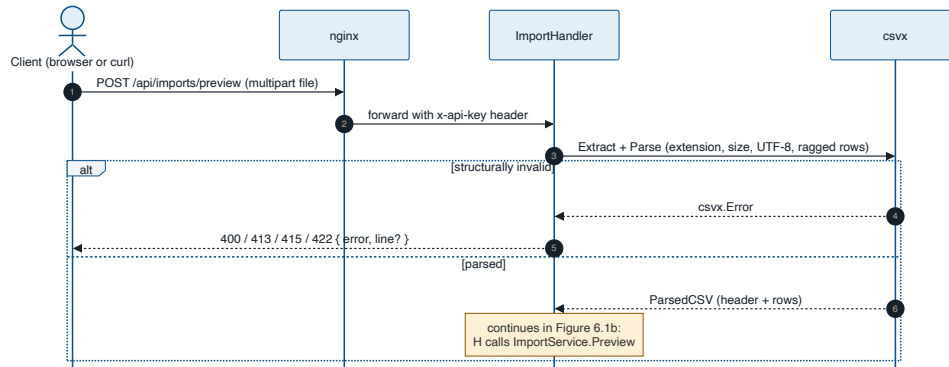


Figure 6.1a — POST /api/imports/preview : upload and structural parse

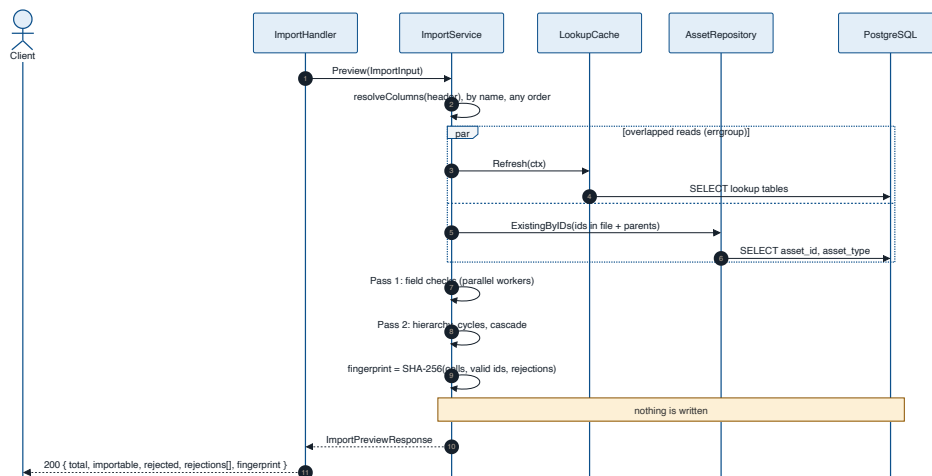


Figure 6.1b — Validation and the preview response

6.2 Confirm an import: revalidate, then commit in one transaction

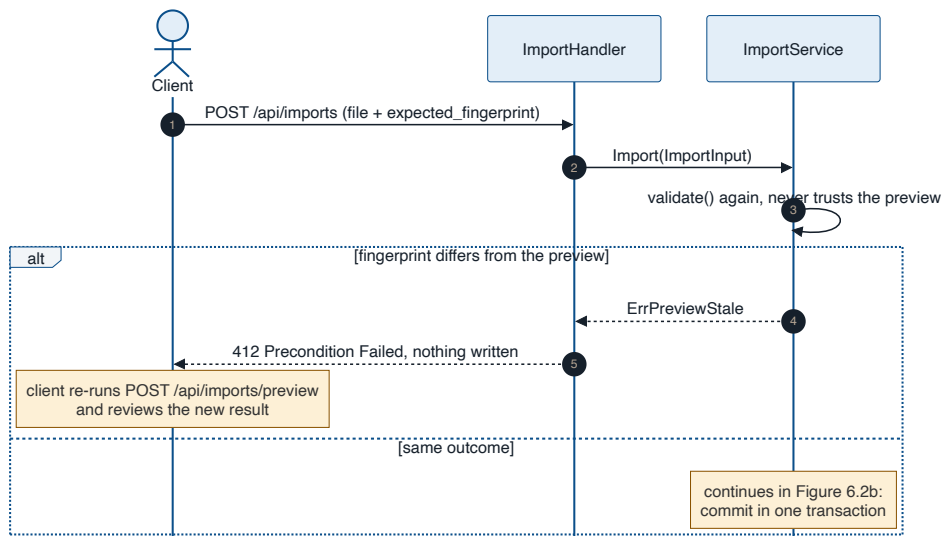


Figure 6.2a — POST /api/imports : revalidation and the 412 path

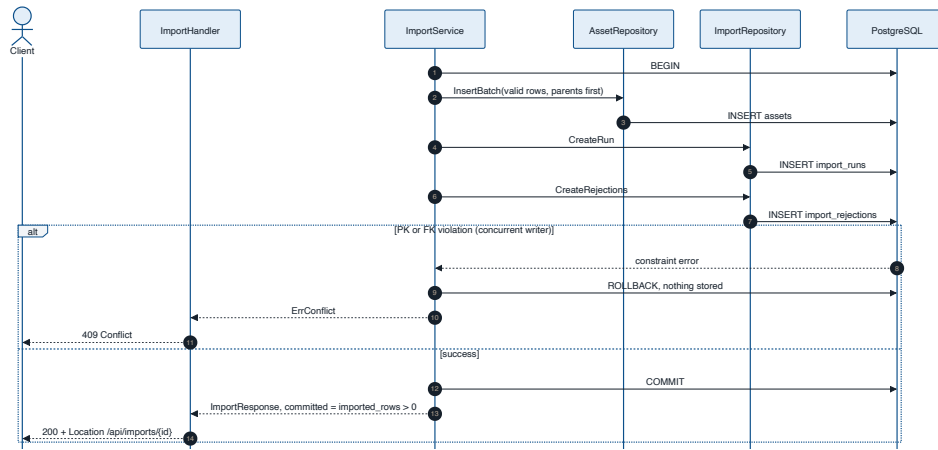


Figure 6.2b — The transaction, including the 409 and success paths

6.3 Search, then locate the asset in the hierarchy

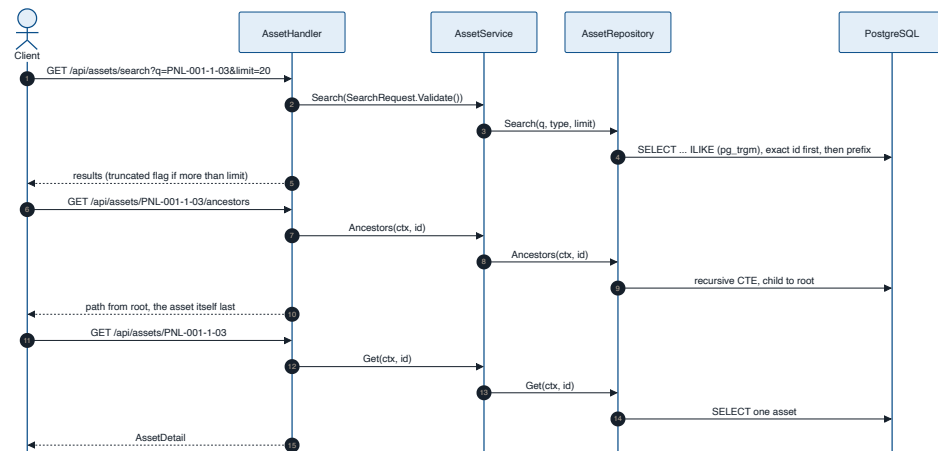


Figure 6.3 — Search, ancestors, details: the three calls behind “open this asset in the tree”

6.4 Browse the hierarchy: roots, then children with counts beneath

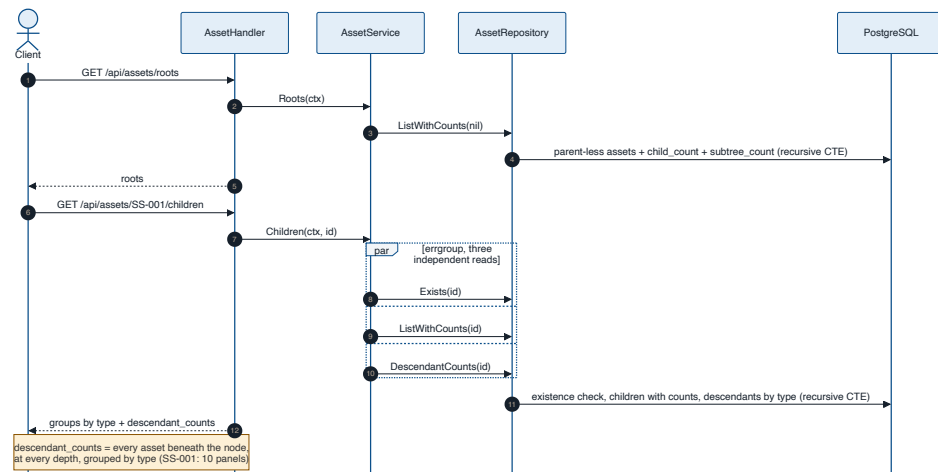


Figure 6.4 — GET /roots and GET /{id}/children, including descendant_counts

6.5 Delete one asset

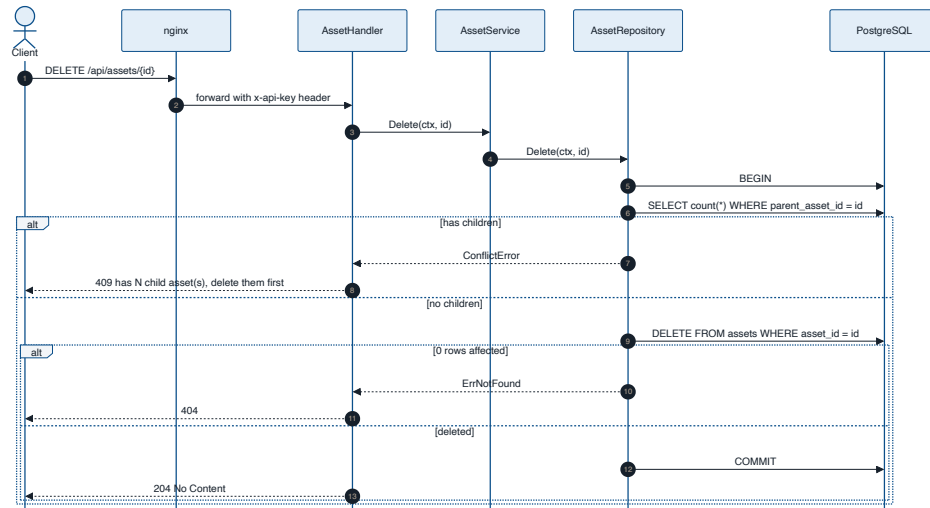


Figure 6.5 — DELETE /api/assets/{id} : count-then-delete in one transaction

6.6 API-key check on every request

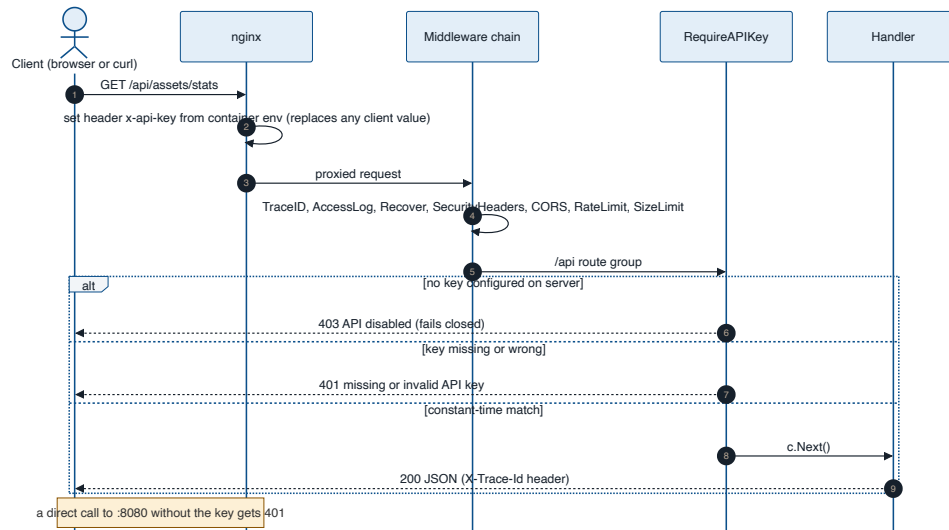


Figure 6.6 — nginx injects the key; the backend fails closed

7. Validation algorithm and import transaction

Row order must not decide validity, and a child may appear before its parent in the same file, so a single top-to-bottom pass cannot work. Validation is two passes over the file, plus a fingerprint so the confirm step can prove it is committing exactly what the user reviewed.

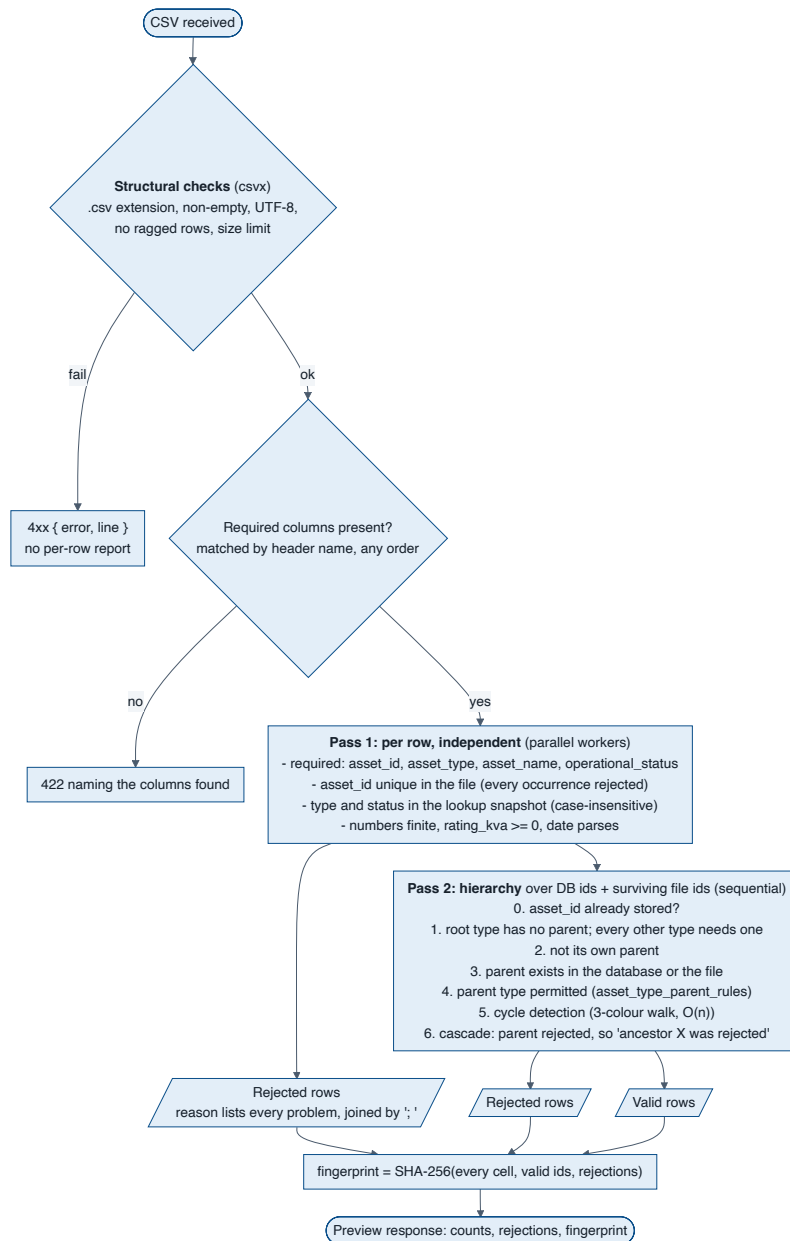


Figure 7.1 — Validation pipeline (`ImportService.validate`)

7.1 Rules and where each is enforced

Pass	Rule	Rejection wording (verbatim from the running system)
1	Required fields present	missing required field: asset_id
1	Unique within file (every occurrence rejected, naming the others)	duplicate asset_id "X" (also on rows 5, 9)
1	Type in lookup set (case-insensitive; stored canonical)	invalid asset_type "GENERATOR" (allowed: LV_BOARD, SUBSTATION, ...)
1	Status in lookup set	invalid operational_status "BROKEN" (allowed: IN_SERVICE, MAINTENANCE, OUT_OF_SERVICE)
1	Numbers finite; rating_kva ≥ 0	rating_kva cannot be negative (-500)

Pass	Rule	Rejection wording (verbatim from the running system)
1	Date parses (YYYY-MM-DD , D/M/YYYY , D/M/YY ; day-first)	commissioned_date is not a valid date ("2026-13-41"; expected YYYY-MM-DD or D/M/YY)
2 / 0	Not already stored	asset_id "SS-001" already exists in the database
2 / 1	Root types have no parent; others need one	SUBSTATION is a root type and must not have a parent_asset_id (got "SS-T01") TRANSFORMER requires a parent_asset_id
2 / 2	Not its own parent	asset cannot be its own parent
2 / 3	Parent exists in DB u surviving file rows	parent "SWB-404" does not exist in the database or file
2 / 4	Parent type permitted	parent "SS-T01" is a SUBSTATION, but a SWITCHBOARD_PANEL must have a parent of type SWITCHBOARD
2 / 5	No cycles	cycle detected: SWB-CY-A -> SWB-CY-B -> SWB-CY-A
2 / 6	Cascade: a valid row under a rejected ancestor is rejected too	ancestor X was rejected (X = the nearest ancestor rejected in its own right, so the message does not depend on row order)

Why cycle detection looks at every in-file parent link

Type rules are themselves acyclic (a switchboard can only sit under a substation), so a cycle such as SWB-CY-A ↔ SWB-CY-B can never pass the parent-type check. Restricted to rows that passed checks 1–4 it would be unreachable and every cycle member would be mislabelled “wrong parent type”. A cycle member’s reason therefore *leads with the cycle* and then appends any other problem the row has (visible on rows 93–96 below).

7.2 Worked example: the supplied sample file

backend/testdata/grid_assets.csv has 222 data rows: 205 valid and 17 deliberately bad. Checked against an empty database, the system produced exactly these 17 rejections (CSV row numbers count the header as row 1):

Row	Asset ID	Reason reported	Rule
91	PNL-M	parent "SWB-404" does not exist in the database or file	2/3
92	SWB-S	parent "SWB-SELF" does not exist in the database or file	2/3
93	SWB-CY-A	cycle detected: SWB-CY-A -> SWB-CY-B -> SWB-CY-A; parent "SWB-CY-B" is a SWITCHBOARD, but a SWITCHBOARD must have a parent of type SUBSTATION	2/5 + 2/4
94	SWB-CY-B	cycle detected: SWB-CY-B -> SWB-CY-A -> SWB-CY-B; parent "SWB-CY-A" is a SWITCHBOARD, but ...	2/5 + 2/4
95	SWB-CY-C	cycle detected: SWB-CY-C -> PNL-CY-C -> PNL-CY-D -> SWB-CY-C; parent "PNL-CY-C" is a SWITCHBOARD_PANEL, but ...	2/5 + 2/4
96	PNL-CY-C	cycle detected: PNL-CY-C -> PNL-CY-D -> SWB-CY-C -> PNL-CY-C; parent "PNL-CY-D" is a SWITCHBOARD_PANEL, but ...	2/5 + 2/4
97	PNL-CY-D	cycle detected: PNL-CY-D -> SWB-CY-C -> PNL-CY-C -> PNL-CY-D	2/5
98	PNL-BP	parent "SS-T01" is a SUBSTATION, but a SWITCHBOARD_PANEL must have a parent of type SWITCHBOARD	2/4
99	TX-BP	parent "SWB-T01" is a SWITCHBOARD, but a TRANSFORMER must have a parent of type SUBSTATION	2/4
178	GEN-001	invalid asset_type "GENERATOR" (allowed: LV_BOARD, SUBSTATION, SWITCHBOARD, SWITCHBOARD_PANEL, TRANSFORMER)	1
179	TX-BD	commissioned_date is not a valid date ("2026-13-41"; expected YYYY-MM-DD or D/M/YY)	1
180	TX-NR	rating_kva cannot be negative (-500)	1
181	PNL-BS	invalid operational_status "BROKEN" (allowed: IN_SERVICE, MAINTENANCE, OUT_OF_SERVICE)	1
182	(blank)	missing required field: asset_id	1
183	PNL-N	missing required field: asset_name	1
184	SS-H-P	SUBSTATION is a root type and must not have a parent_asset_id (got "SS-T01")	2/1
185	TX-N-P	TRANSFORMER requires a parent_asset_id	2/1

Reading row 92 accurately

Row 92 (`SWB-S` → `SWB-SELF`) looks like it was meant to be a self-parent test, but its parent id differs from its own id, so the system correctly reports a *missing parent* rather than a self-parent. The self-parent rule (2/2) is exercised by a unit test in `service/import_test.go` ("asset is its own parent"), not by this sample file.

Uploading the same file a second time, against the now-populated database, rejects all 222 rows: the 205 already-stored ids as duplicates (rule 2/0) and the same 17 for their original reasons. This was observed on the running system and confirms that **existing assets are never updated or duplicated**.

7.3 Import outcome: check → review → confirm (FR-1.11)

The brief allows all-or-nothing or partial import. The first design was all-or-nothing, but the supplied file contains 17 bad rows, so it would have committed nothing. The chosen design is **partial import behind a mandatory-in-the-UI preview**, which makes the outcome predictable and leaves the choice to the user:

- 1 **Preview** (`POST /api/imports/preview`) runs both passes against current data and returns counts, every rejection and a fingerprint. It writes nothing: no assets, no `import_runs` row, no locks.
- 2 The user cancels (fix the file) or **confirms**. `POST /api/imports` re-uploads the same file with the fingerprint and validates **again**; a client-supplied outcome is never trusted.
- 3 In **one transaction** the accepted rows (parents first) and the `import_runs / import_rejections` audit rows are inserted. Rejected rows, and anything under a rejected parent, are skipped. `committed = imported_rows > 0`, and `imported + rejected = total`.
- 4 The fingerprint is a SHA-256 over every cell, the ids that would be stored and every rejection. If the file *or the stored data* changed in a way that alters the outcome, the commit is refused with **412**, nothing is written, and the UI re-checks the file and shows the new result.
- 5 The remaining window between the second validation and the INSERT is closed by the database: a primary-key or FK violation aborts the transaction and is answered with **409**.

There is deliberately no staging table: the browser keeps the file and re-uploads it (capped at `MAX_REQUEST_BYTES`), which avoids server-side state, expiry and cleanup. A caller that omits `expected_fingerprint` (curl, tests) simply commits the valid rows directly.

7.4 Flexible input handling

- **Columns are found by header name, never position.** Headers are normalised (trim, BOM, case, and runs of space/ - / _ fold together, so `Asset ID = asset_id`). Any order works; unknown columns are ignored and echoed back as `ignored_columns`; optional columns may be absent.
- **The schema is one declarative table** (`assetFields` in `service/import.go`): name, required-ness, and a check/apply pair per column. Adding a column is one entry plus a model field and DB column.
- **Rows are unbounded** by design; the only ceiling is the upload size. Fully blank rows (Excel's trailing `, , , ,`) are skipped. Ragged rows are a structural 422 naming the line, because an unquoted comma would silently shift every later column.
- UTF-8 BOM, CRLF, quoted multi-line cells and surrounding whitespace are handled; `csv_row` always stays the true file line.

8. API reference

The generated OpenAPI spec (`backend/docs/` , served at `/swagger/index.html` , `doc.json` and `swagger.yaml`) is the contract; this chapter records the decisions behind it. All JSON keys are `snake_case` via explicit tags. Every `/api` endpoint requires `x-api-key` .

8.1 Endpoints

Method	Path	Purpose
POST	<code>/api/imports/preview</code>	Multipart CSV (field <code>file</code>) → two-pass validation → counts, rejections, <code>fingerprint</code> . Writes nothing.
POST	<code>/api/imports</code>	Multipart CSV (+ optional <code>expected_fingerprint</code>) → validate again → commit valid rows in one transaction. 412 if the fingerprint no longer matches.
GET	<code>/api/imports</code>	Import history, newest first (<code>limit</code> , <code>offset</code>).
GET	<code>/api/imports/template</code>	Downloadable CSV template; header generated from the importer's own column table, plus four example rows.
GET	<code>/api/imports/{importId}</code>	A past import's summary and rejected rows.
GET	<code>/api/lookups</code>	Asset types, statuses and parent-type pairings validation runs against.
GET	<code>/api/assets</code>	All assets, paged; <code>q</code> , <code>type</code> , <code>status</code> , <code>sort</code> (<code>asset_id name type status parent</code>), <code>dir</code> . Sorted in SQL from a fixed column list, ties by id, roots last when sorting by parent.
GET	<code>/api/assets/stats</code>	Counts by asset type and by operational status.
GET	<code>/api/assets/roots</code>	Top-level (parent-less) assets with <code>child_count</code> and <code>subtree_count</code> .
GET	<code>/api/assets/search</code>	<code>q</code> (1–200 chars), optional <code>type</code> , <code>limit</code> (default 50, max 200). Case-insensitive substring on id or name; exact id first, then id prefix, then id order. <code>%</code> and <code>_</code> are literal.
GET	<code>/api/assets/{assetId}</code>	One asset's full details.
DELETE	<code>/api/assets/{assetId}</code>	Delete one asset; 409 if it still has children; 404 if unknown; 204 on success.
GET	<code>/api/assets/{assetId}/children</code>	Immediate children grouped by type (with subtree counts) plus <code>descendant_counts</code> per type.
GET	<code>/api/assets/{assetId}/ancestors</code>	Path from the top of the hierarchy to the asset, the asset itself last.
GET	<code>/healthz</code> , <code>/swagger/*</code> , <code>/docs</code>	Open (no key): health check with DB status; Swagger UI and spec.

8.2 Status codes and error body

Every failure has the same shape, `{"error": "...", "line"? : n, "trace_id"? : "a1b2c3d4"}` . What is safe to show a client (bad extension, missing column, malformed line) is returned verbatim because it tells the user how to fix the request; anything unexpected becomes a generic 500 carrying only the trace ID, and the real error is in the server log under the same ID.

Status	When
200	Import processed. Both outcomes (data committed or not) are 200 with the same shape and a <code>Location: /api/imports/{id}</code> header; <code>committed</code> is the discriminator.
204	Asset deleted.
400	No file, empty file, invalid query/parameter (e.g. unknown <code>type</code> on search, non-numeric <code>limit</code>).
401 / 403	API key missing or wrong / no key configured on the server (fails closed).
404	Unknown asset or import id; unknown path (<code>{"error": "not found"}</code>).
405	Wrong verb, with an <code>Allow</code> header.
409	Delete of an asset that has children, or another writer collided with the commit (PK/FK violation, nothing stored).
412	Confirm refused: the file or stored data changed since the preview. Nothing written.
413 / 415	Body over <code>MAX_REQUEST_BYTES</code> / file is not <code>.csv</code> .
422	Malformed CSV (with <code>line</code>), header missing a required column or repeating a known one, or a header with no data rows.
429	Rate limit (100 requests / 10 s per client and path).
503	The request exceeded its time budget (import: 2 minutes) and was cancelled.

8.3 Response shapes worth knowing

Import result (`imported_rows + rejected_rows = total_rows`; `rejections` lists every skipped row, including rows skipped because their parent was rejected):

```
{
  "import_id": "uuid",
  "total_rows": 222,
  "imported_rows": 205,
  "rejected_rows": 17,
  "committed": true,
  "ignored_columns": [],
  "rejections": [
    { "csv_row": 180, "asset_id": "TX-NR", "reason": "rating_kva cannot be negative (-500)" },
    { "csv_row": 93, "asset_id": "SWB-CY-A", "reason": "cycle detected: SWB-CY-A -> SWB-CY-B -> SWB-CY-A; ..." }
  ]
}
```

Children of a substation (real response for `SS-001`). Note that `descendant_counts` carries the per-type counts *beneath* the node, including the 10 switchboard panels two levels down. The details panel shows these as its “Contents by type, at every level” cards (FR-3.4):

```
{
  "asset_id": "SS-001", "total_children": 5,
  "groups": [ { "asset_type": "LV_BOARD", "count": 1, "subtree_count": 1 },
               { "asset_type": "SWITCHBOARD", "count": 2, "subtree_count": 12 },
               { "asset_type": "TRANSFORMER", "count": 2, "subtree_count": 2 } ],
  "descendant_counts": [ { "asset_type": "LV_BOARD", "count": 1 },
                         { "asset_type": "SWITCHBOARD", "count": 2 },
                         { "asset_type": "SWITCHBOARD_PANEL", "count": 10 },
                         { "asset_type": "TRANSFORMER", "count": 2 } ]
}
```

Other shapes. `AssetSummary` = id, parent id (null at the top), type, name, status. `AssetDetail` adds voltage, rating, manufacturer, model, serial, commissioned date (YYYY-MM-DD) and timestamps; nullable columns are `null`. `AssetNode` = summary + rating + commissioned date + `child_count` + `subtree_count` (including itself; a leaf is 1). A childless asset returns empty lists from `/children`, not 404.

8.4 Trying it with curl

```
KEY=$(grep ^API_KEY .env | cut -d= -f2)

curl -H "x-api-key: $KEY" http://localhost:8080/api/assets/stats
curl -H "x-api-key: $KEY" -F file=@backend/testdata/grid_assets.csv \
  http://localhost:8080/api/imports/preview          # writes nothing
curl -H "x-api-key: $KEY" -X DELETE http://localhost:8080/api/assets/SS-001
# -> 409 {"error":"asset \"SS-001\" has 5 child asset(s); delete them first","trace_id":"..."}
curl -i http://localhost:8080/api/assets/stats        # no key -> 401
curl http://localhost:8080/healthz                    # {"status":"ok","db":"up"}
```

The 401, 409 and healthz responses above were observed on the running system while preparing this document.

9. Frontend implementation

9.1 Routes and layouts

Route	Layout	Content
/	ExplorerLayout (tree rail)	Overview: totals by type, top-level assets, status donut. First-run empty state guides to Import.
/assets	ExplorerLayout	All assets: filter, sort, paging; state kept in the URL.
/assets/:assetId	ExplorerLayout	Asset details; tree expands to and highlights the asset.
/import	PageLayout (no rail)	Upload card, review/report, import history. <code>/imports</code> redirects here.
*	ExplorerLayout	Not-found panel.

`AppRoot` owns the client state and wraps both layouts, so the import flow (including the chosen `File`) survives navigation between pages.

9.2 The import flow as a state machine

`useImportFlow` owns the stages. A structurally bad file keeps the file selected so it can be swapped; a 412 on confirm silently re-checks the same file and shows the new result with a notice; a 409 offers "Check again".

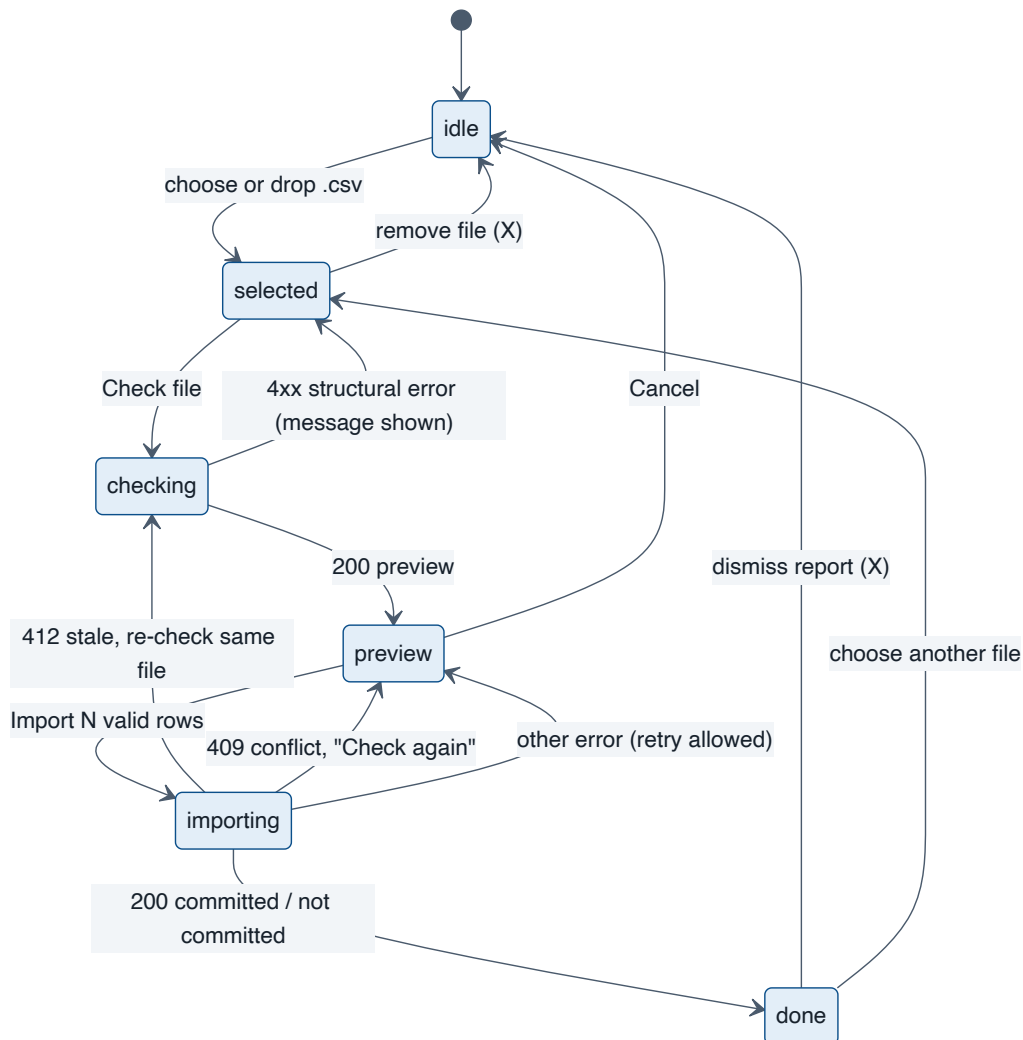


Figure 9.1 — Import flow stages (`state/explorerContext.ts`, `state/useImportFlow.ts`)

9.3 Server state, caching and generated types

- **TanStack Query only** for server state; keys and fetchers live in `src/api/queries.ts` (`roots`, `asset`, `children`, `ancestors`, `search`, `assetList`, `stats`, `lookups`, `previewImport`, `commitImport`, `importList`, `importDetail`, `deleteAsset`). After a commit or delete, asset

and import queries are invalidated so every panel refreshes.

- **Types are generated:** `npm run generate:types` converts the backend's Swagger 2 spec to OpenAPI 3 and produces `schema.ts`; `npm run check:types` fails if it is stale.
- **Client state** is limited to the expanded-node set (a pure reducer with `toggle`, `expand`, `collapse`, `expandMany`, `collapseAll`), the import flow, the mobile drawer, and the theme.

9.4 Design tokens, themes and layout

- **Tokens only.** Colours, radii and the font (Inter, self-hosted) are defined once in `index.css`; components use utilities such as `bg-surface` and `text-text-muted`, never hex codes. Dark mode redefines the same token names under `:root[data-theme="dark"]`; new colours must be added to both and pass WCAG AA (4.5:1). A test enforces the spacing scale.
- **Theme** follows the OS on first visit and is remembered in the browser afterwards.
- **Mobile-first.** Below the `md` breakpoint the rail becomes a Radix Dialog drawer that closes on navigation; wide tables scroll sideways and keep the Asset ID on one line.

9.5 Contents by type (FR-3.4)

`AssetDetailsPanel` shows one row of cards, **Contents by type, at every level**, and under it the immediate-children table. Each card is one asset type that *can* occur beneath the asset: the big number is how many sit anywhere beneath it, at every depth (from `descendant_counts`), and the line under it says how many of those are immediate children (from `groups`), for example “*0 direct · 10 deeper*” for the switchboard panels of a substation. A type with none shows 0 and “None”, so a substation always shows transformers, LV boards, switchboards and switchboard panels. The immediate-children table lists the direct children and sorts by name, type, status, rating or commissioned date.

Which types can occur beneath an asset is worked out by `descendantTypes()` (`src/lib.ts`): it follows the backend's parent rules (`GET /api/lookups`) child-to-parent, transitively, so no type name is written in the code and a new type or pairing added to the lookup tables appears with no frontend change. Cards and table share one `GET /children` request. An asset with no children shows the empty message instead; if the request fails, the section explains the failure and offers **Try again**; if only the lookups fail, the cards fall back to the types the API returned.

9.6 Per-panel states and accessibility

- Tree, details, search and import each render their own **loading skeleton, empty state and error state with a retry**; there is no global spinner (Fig. 13.1, 13.17).
- **Tree semantics:** `role="tree" / treeitem`, `aria-expanded`, `aria-selected` kept in sync; each row has an `aria-labelledby` pointing at its own label so `aria-owns` does not pull its whole subtree into its accessible name.
- **Search** is an ARIA combobox with a listbox; **keyboard** operable throughout; `:focus-visible` is never suppressed; destructive actions use a focus-trapping modal.

9.7 Asset-type icons and the footer

Icons. One map in `src/api/assetTypes.ts` gives each asset type a label, a short code and an icon (lucide), reused by the tree, the type badges, the details, the overview and search, so an asset looks the same everywhere. An unknown type from the lookup table falls back to a derived label and a generic box icon. The LV board, switchboard and switchboard panel icons were replaced in v1.2 because the previous ones did not suggest the equipment; the substation and transformer icons are unchanged.

Footer. The footer is the end of the page, not a bar fixed to the screen. `AppVersion` (app name, version and copyright) sits at the bottom of the tree rail, and at the bottom of the mobile drawer; `AppFooter` is the last element of the scrolling main panel and holds the **API docs** (`/swagger/index.html`, new tab) and **Health** (`/healthz`) links and a clock. The clock ticks every second in the browser's own time zone with its abbreviation, and is deliberately not an `aria-live` region so a screen reader is not interrupted. A page without a rail (Import) shows the version at the left of its footer. The version is `version` in `frontend/package.json`, injected at build time by `vite.config.ts` as `__APP_VERSION__`, so it is changed in one place; it is the application's version, separate from the API's `@version 1.0` in the Swagger spec. Only design tokens are used, so dark mode needs no extra colours.

10. Testing and verification

10.1 Strategy

Level	What is covered
Validator unit tests	Table-driven: one case per rejection category (missing field, duplicate id, bad type, bad status, negative rating, bad date, substation with parent, orphan, missing parent, self-parent, wrong parent type, cycle, cascade), plus header resolution, ordering and fingerprint.
Repository / service integration	Real PostgreSQL. <code>testutil.NewTestDB</code> creates an isolated schema per test with migrations applied and drops it afterwards, so tests are parallel-safe and never touch dev data. Covers the recursive CTEs, commit vs all-rejected, dynamic lookups, concurrent identical uploads and the full HTTP stack through <code>router.New</code> .
Swagger guards	Every route in <code>router.New</code> is in the spec and vice versa, every <code>\$ref</code> resolves, UI and spec are served without a key. A route or DTO change without <code>make docs</code> fails the build.
Middleware	API key (401/403/constant-time), trace-ID echo, panic recovery.
Frontend	Vitest + React Testing Library with <code>fetch</code> stubbed (<code>mockApi</code> , <code>renderApp</code> , asserting call order; no MSW): tree expand, search-select-and-highlight, import flow, history, donut, theme, reducer, spacing scale.

10.2 Results measured while preparing this document

Backend	<code>go test -race -count=1 ./...</code> with <code>TEST_DATABASE_URL</code> set (so integration tests ran): all 13 test packages ok ; 119 test functions, 286 including subtests; 0 failed, 0 skipped .
Frontend	<code>npm test</code> : 15 files, 112 tests, all passed ; <code>oxlint</code> , <code>check:types</code> and <code>npm run build</code> clean. They include the footer (version, copyright, link targets, the clock ticking and being cleared on unmount, and where the version appears with and without a rail).
End-to-end (manual)	Against a clean, isolated Compose stack: import of the supplied file → 222 total / 205 imported / 17 rejected; totals by type 13/30/16/25/121; status split 130/37/38; re-upload rejects all rows; delete of a parent → 409; request without key → 401. Screenshots in chapter 13 were retaken in this run, on a clean stack started with <code>docker compose -p manualshots</code> on ports 18080 and 18081.

What is not tested

There is no automated browser end-to-end test and no screen-reader test; the screenshots here are a manual pass in headless Chrome, not a regression suite. There is no CI pipeline, so nothing runs the tests automatically: they are run by hand before a release (chapter 11). See chapter 14.

11. Build and deployment (local)

Only a local build and run is needed, so there is **no CI/CD pipeline**. An earlier CI pipeline (tests on every push, build and an SSH deploy on a tag) was written but never ran, because no runner existed; it was removed on 21 September 2026 together with its variables script. Tests are run by hand, and “deploying” means the commands below. The same commands, with more troubleshooting, are in `docs/DEPLOYMENT.md` in the repository.

11.1 Running the whole stack

```
cp .env.example .env          # set API_KEY: openssl rand -hex 32
docker compose up --build     # db -> migrate -> backend -> frontend (add -d to run in the background)

Frontend  http://localhost:8081
Health    http://localhost:8081/healthz          (also :8080/healthz)
API docs  http://localhost:8081/swagger/index.html (linked in the app footer; also :8080/swagger/index.html)
```

Health checks poll every 2 s while a service starts (so `depends_on: service_healthy` is quick) and then every 5 minutes; `db`, `backend` and `frontend` restart automatically (`restart: unless-stopped`). To reset all data: `docker compose down -v` (there is deliberately no delete-all in the app). After an upgrade that edits the initial migration, run `docker compose down -v` first because `migrate` will not re-apply a version it recorded as applied.

11.2 Developer commands

```
cd backend && go vet ./... && go test -race ./...      # DB tests skip unless TEST_DATABASE_URL is set
cd backend && make docs                               # regenerate Swagger after annotation/DTO changes
cd frontend && npm run generate:types                  # schema.ts from backend/docs/swagger.json
cd frontend && npm run check:types                     # fails if schema.ts is stale
cd frontend && npm test && npm run lint && npm run build
```

11.3 Everyday and troubleshooting commands

Command	Use
<code>docker compose ps</code>	State and health of each service.
<code>docker compose logs -f backend</code>	Live log; every line has a <code>tid=</code> trace id, the same as the <code>X-Trace-Id</code> header.
<code>docker compose up --build -d</code>	Rebuild changed images and restart (after <code>git pull</code>).
<code>docker compose down / down -v</code>	Remove containers / also delete the database volume (all data lost).
<code>lsof -nP -iTCP:8081 -sTCP:LISTEN</code>	Who holds a port that is “already allocated”; or change the port in <code>.env</code> .
<code>docker compose exec db sh -c 'psql -U "\$POSTGRES_USER" -d "\$POSTGRES_DB"'</code>	Look inside the database (<code>select version, dirty from schema_migrations;</code>).
<code>docker inspect --format '{{.State.Health.Status}}' <container></code>	Exact health state of one container.

Symptoms: every `/api` call 403 means `API_KEY` is empty in `.env`; 401 means the frontend and backend were started with different keys (recreate both). A 502 from nginx means the backend is down or restarting.

12. Security and operations

Topic	Behaviour
API key	<code>x-api-key</code> header only (never query string or cookie). Constant-time compare. 401 wrong/missing; 403 if the server has no key (the app is unusable without <code>API_KEY</code> , by design).
Where the key lives	In nginx's config inside the frontend container and in the Vite dev proxy. It is never in the browser or JS bundle. Consequence: anyone who can use the UI can use every feature, including delete; the key protects the backend port from direct callers, not the UI from its users.
Transport	No TLS in the Compose setup; anywhere beyond a local machine the key needs TLS in front.
Rate limiting	100 requests / 10 s per client and path, per process. A burst of uploads from one address can hit 429.
Hardening	Security headers, docs-specific CSP, request-size cap, server read/write/idle timeouts against slow-loris, graceful shutdown, panic recovery.
Tracing and logs	Every request gets an 8-hex <code>X-Trace-Id</code> (a client-sent one is echoed unchanged) that appears in every log line and error body. Rejected rows are logged with <code>import_id</code> , <code>file</code> , <code>line</code> , <code>asset_id</code> , <code>pass</code> , <code>reason</code> . Values are quoted so user-controlled text cannot forge log lines.
Audit	<code>import_runs</code> and <code>import_rejections</code> record every import (filename, client IP, counts, committed flag). Deletions are logged but not recorded in the database.
Backup / restore	Not built in. Data lives in the <code>pgdata</code> Docker volume; use <code>pg_dump</code> against the <code>db</code> container for backups.

13. User manual

This chapter is for the people who use the system day to day. Every screenshot was captured from the running application using the supplied sample file, `grid_assets.csv`. No technical knowledge is needed.

13.1 Before you start

- Open the application in a current browser at **http://localhost:8081** (or the address your administrator gives you). There is **no login**: the system is protected at the server, so you do not type a password or key.
- The first time, the system is empty. It shows the guidance in Fig. 13.1 until the first import.
- You need a CSV file of assets. Download a template from the **Import** page (see 13.3).

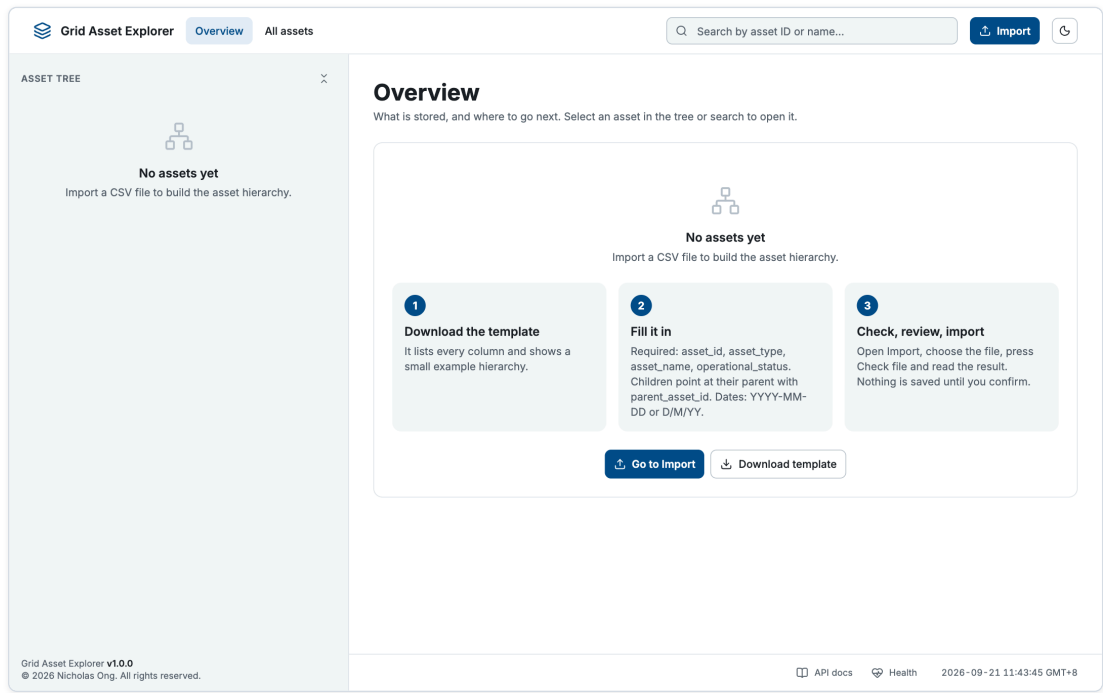


Figure 13.1 — First run: the Overview and the asset tree both explain that there are no assets yet, and how to add some.

13.2 The screen at a glance

Every page shares one top bar. The explorer pages add a **tree** on the left; the Import page uses the full width.

Control (top bar)	What it does
Grid Asset Explorer logo	Identifies the application.
Overview	Fleet totals, top-level assets and the status split.
All assets	A filterable, sortable table of every asset.
Search box	Find an asset by ID or name from any page.
Import button	Go to the CSV import page.
Moon / sun button	Switch between light and dark themes.

13.3 Importing assets from a CSV file

Importing is a deliberate three-step process: **check** the file, **review** the result, then **confirm**. **Nothing is saved until you confirm.**

Step by step

- 1 **Open the Import page** with the **Import** button. On the left is the upload card; on the right is the import history (Fig. 13.2). On a phone they are stacked.
- 2 **Get a template (optional).** Choose **Download template**. It contains every column and four example rows forming a small valid hierarchy.

- 3 **Choose your file.** Drag a `.csv` onto the dashed area, or press **Choose file**. The dropzone is replaced by the file's name and size (Fig. 13.3). Use the **X** to remove it and pick another.
- 4 **Press Check file.** The file is validated on the server. **Nothing is stored.**
- 5 **Read the review** (Fig. 13.5). The banner says *Nothing has been saved yet*, and three tiles show **Total rows**, **Can be imported** and **Will be rejected**. Below is the list of rejected rows: the row number in your file, the asset ID, and the reason.
- 6 **Decide.** Either
 - **Import N valid rows** — stores the valid rows; rejected rows are skipped, or
 - **Cancel** — nothing is saved; fix the file and check it again.

Download report (CSV) saves the rejected rows so you can work through them in a spreadsheet.

- 7 **Read the result** (Fig. 13.6). A banner states explicitly whether data was committed, for example *"Data committed: yes — 205 rows stored, 17 rejected rows were not saved"*. The import also appears at the top of the history, marked *Just imported*.

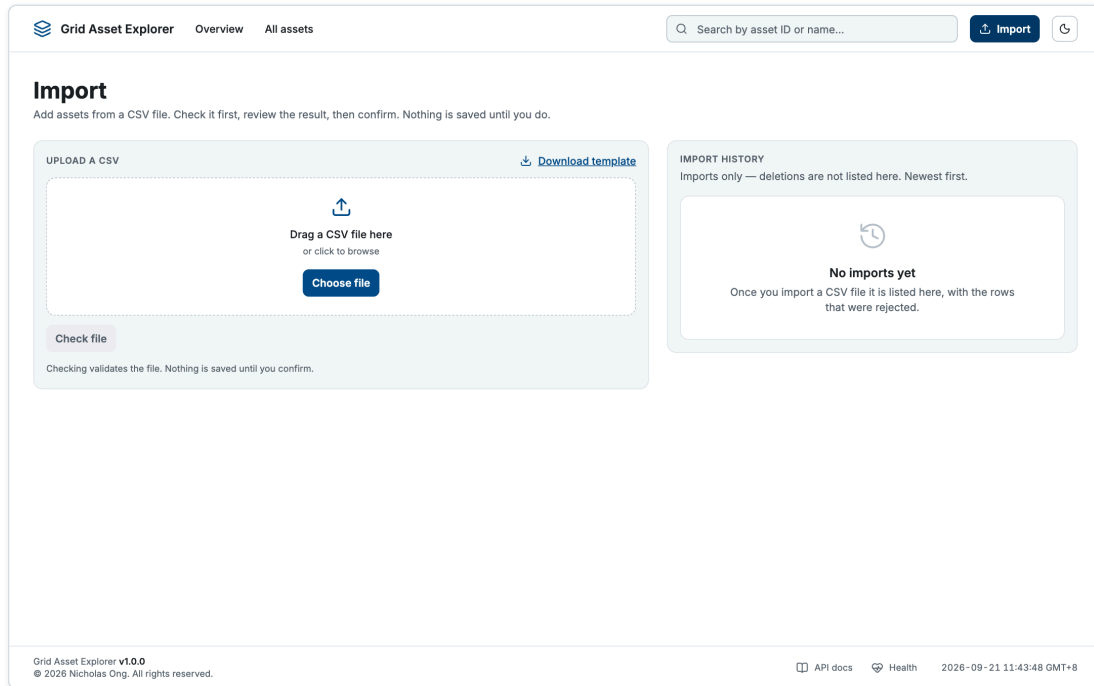


Figure 13.2 — Import page, ready for a file

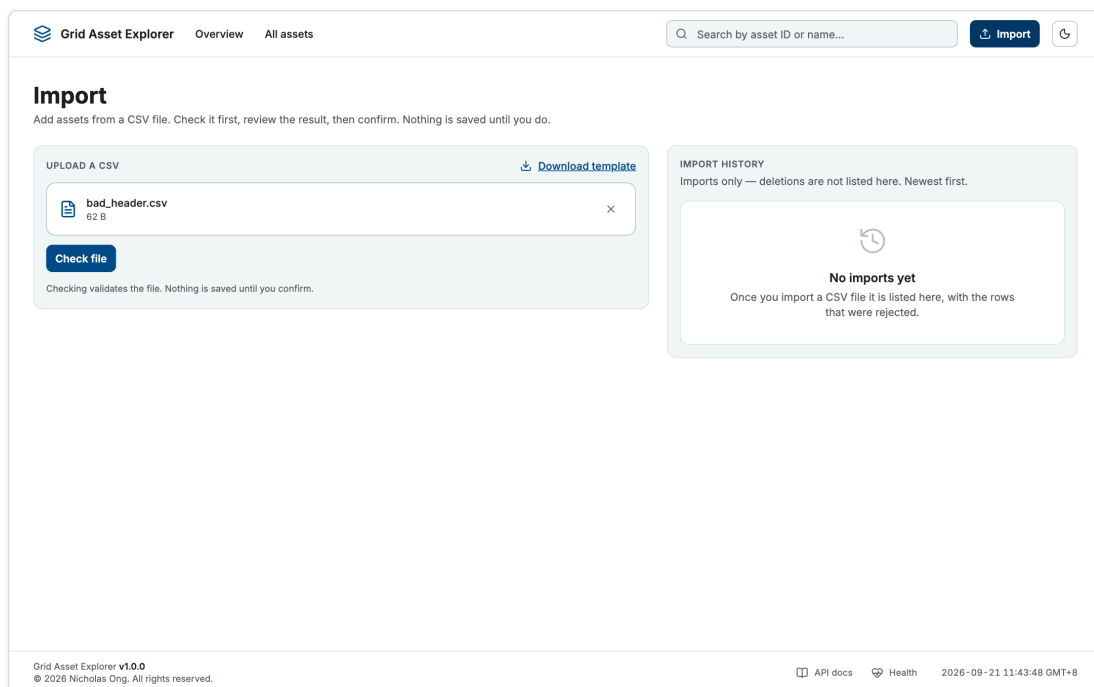


Figure 13.3 — File chosen; **Check file** is now active

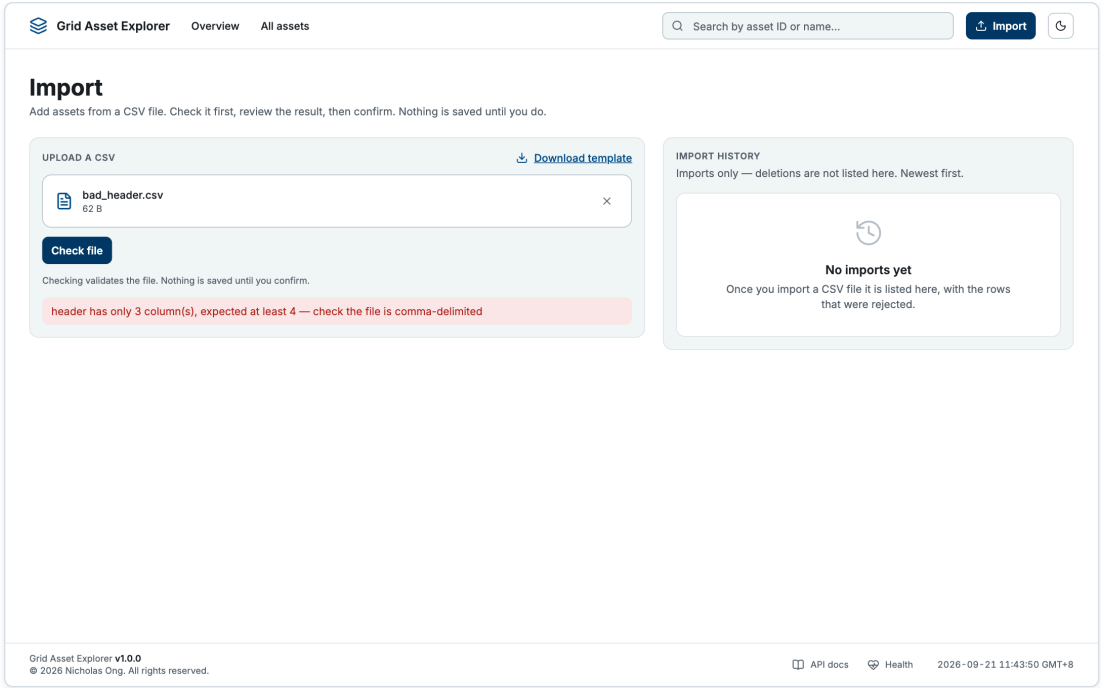


Figure 13.4 — A file that cannot be processed at all (here, only three columns) shows a red message; the file stays selected so you can swap it.

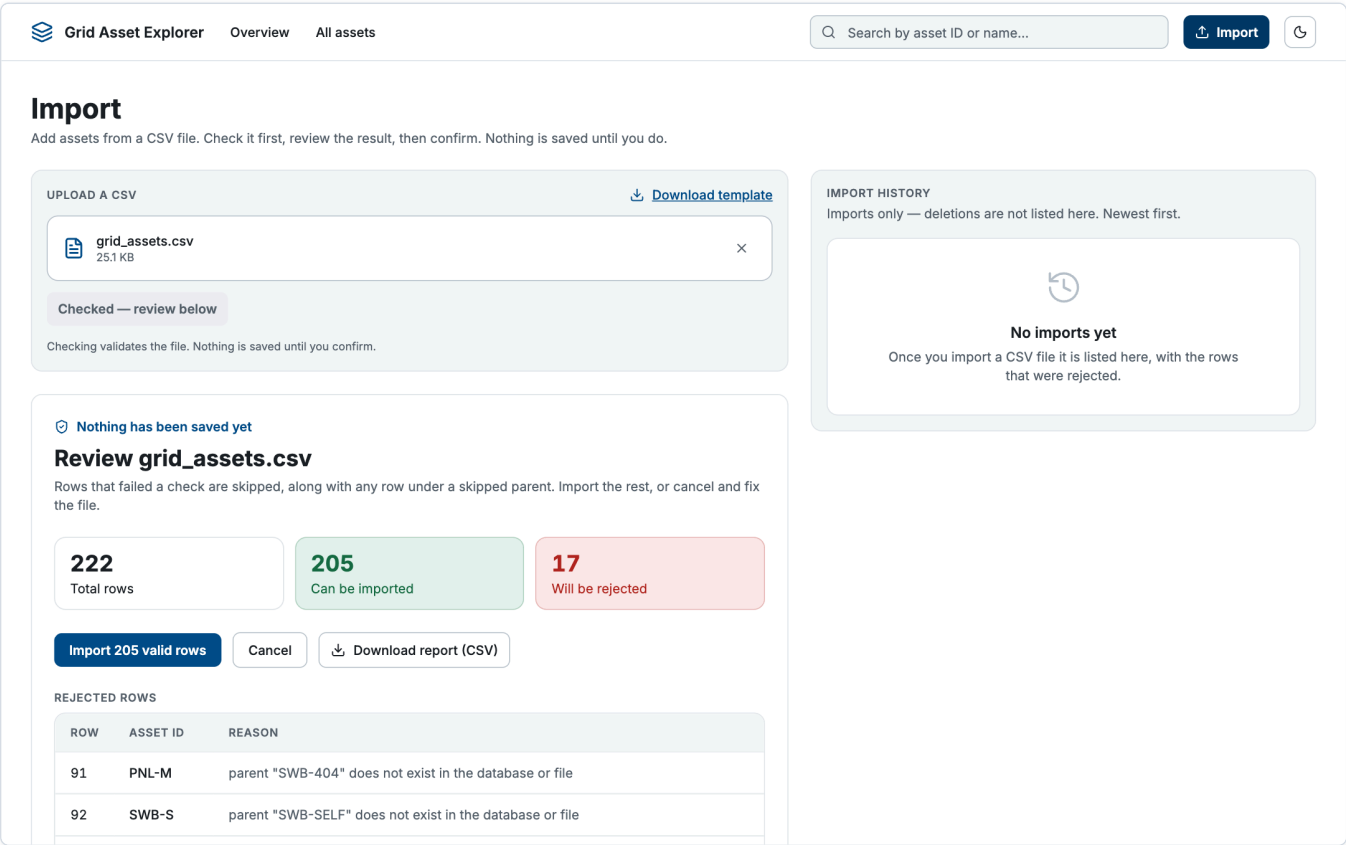


Figure 13.5 — Review: 222 rows, 205 can be imported, 17 will be rejected. Nothing is saved yet. The rejected-rows list is a scrolling region; the first rows are visible and the rest scroll.

Import

Add assets from a CSV file. Check it first, review the result, then confirm. Nothing is saved until you do.

UPLOAD A CSV [Download template](#)

Drag a CSV file here
or click to browse

Choose file

Check file

Checking validates the file. Nothing is saved until you confirm.

IMPORT HISTORY
Imports only — deletions are not listed here. Newest first.

> [grid_assets.csv](#) **Partly stored**

205 of 222 rows imported · 17 rejected
21 Sept 2026, 11:43 **Just imported**

Showing 1-1 of 1 imports < Previous Page 1 of 1 Next >

Data committed: yes — 205 rows stored, 17 rejected rows were not saved

grid_assets.csv

222 Total rows

205 Imported

17 Rejected

REJECTED ROWS [Download report \(CSV\)](#)

ROW	ASSET ID	REASON
91	PNL-M	parent "SWB-404" does not exist in the database or file
92	SWB-S	parent "SWB-SELE" does not exist in the database or file

Figure 13.6 — After confirming: “Data committed: yes”, the counts, the rejected rows, and the new entry in the history (“Partly stored”, “Just imported”).

Preparing the CSV

Column	Required	Notes
asset_id	Yes	Unique. Not already stored, and not repeated in the file.
asset_type	Yes	SUBSTATION , TRANSFORMER , LV_BOARD , SWITCHBOARD or SWITCHBOARD_PANEL (any letter case).
asset_name	Yes	Free text.
operational_status	Yes	IN_SERVICE , MAINTENANCE or OUT_OF_SERVICE .
parent_asset_id	Conditional	Blank for a substation; required for everything else. The parent may be already stored or anywhere in the same file , before or after the child.
voltage_kv	No	A number.
rating_kva	No	A number, zero or greater.
manufacturer , model , serial_number	No	Free text.
commissioned_date	No	YYYY-MM-DD , D/M/YYYY or D/M/YY . Always day first : 12/3/10 is 12 March 2010.

Columns are matched by **heading name**, in any order, and headings are forgiving (Asset ID works as asset_id). Extra columns are ignored and reported. **Who may be whose parent:** a transformer, LV board or switchboard belongs to a **substation**; a switchboard panel belongs to a **switchboard**; a substation has no parent.

Understanding rejection messages

Message (abridged)	Meaning and what to do
missing required field: asset_id / asset_name	The cell is empty. Fill it in.
invalid asset_type "GENERATOR" (allowed: ...)	Not a supported type. Use one from the list.
invalid operational_status "BROKEN" (allowed: ...)	Not a supported status.

Message (abridged)	Meaning and what to do
rating_kva cannot be negative (-500)	Ratings must be zero or more.
commissioned_date is not a valid date (...)	Use YYYY-MM-DD or day-first D/M/YY .
duplicate asset_id "X" (also on rows ...)	The same ID is on several rows. All of them are rejected because the system cannot know which is right; keep one.
asset_id "X" already exists in the database	That asset is already stored. Assets are never overwritten; remove it from the file, or delete the stored asset first.
SUBSTATION is a root type and must not have a parent_asset_id	Clear the parent cell.
TRANSFORMER requires a parent_asset_id	Add the parent's ID.
parent "P" does not exist in the database or file	Add the parent to the file (or import it first), or correct the ID.
parent "P" is a SUBSTATION, but a SWITCHBOARD_PANEL must have a parent of type SWITCHBOARD	Wrong kind of parent. See the parent rules above.
cycle detected: A -> B -> A	Assets point at each other in a loop. Break the loop so every chain ends at a substation.
ancestor X was rejected	This row is fine but its parent (or a higher ancestor) was rejected. Fix X and this row is accepted next time.

If the data changes while you are reviewing

If someone else imports or deletes assets after you checked your file, pressing **Import** re-checks the file and shows an updated result with a notice (*"The data changed since you checked the file..."*). Nothing is saved until you review and confirm the new result. If two imports collide at the very last moment you see *"Nothing was saved because the data changed while importing"* with a **Check again** button; again, nothing is half-saved.

Import history

The right-hand card lists every past import, newest first: file name, *N of M rows imported*, how many were rejected, when, and a badge (*Partly stored* when some rows were skipped). Open an entry to see where it was uploaded from and its rejected rows (Fig. 13.7). History lists imports only; deletions are not shown.

The screenshot shows the 'Import' section of the Grid Asset Explorer. At the top, there's a search bar and an 'Import' button. Below, the 'Import' card shows the upload process for 'grid_assets.csv'. It indicates that 222 total rows were processed, with 205 imported and 17 rejected. A table of rejected rows is displayed, showing reasons such as 'parent \"SWB-404\" does not exist in the database or file' and 'cycle detected'. The right-hand card shows the 'Import History' for the same file, indicating it was 'Partly stored' on 21 Sept 2026. The interface includes a search bar, 'Import' button, and navigation tabs like 'Overview' and 'All assets'.

Figure 13.7 — An import opened in the history, showing its rejected rows. On narrower screens the rejected-rows table scrolls sideways.

13.4 The Overview page

Once assets exist, **Overview** shows: the **total** and a count for each asset type; a card for every **top-level asset** (substation) with the number of assets beneath it (click to open); and an **operational status** donut with exact counts and percentages. Hover a slice or a legend row to see its count in the centre.

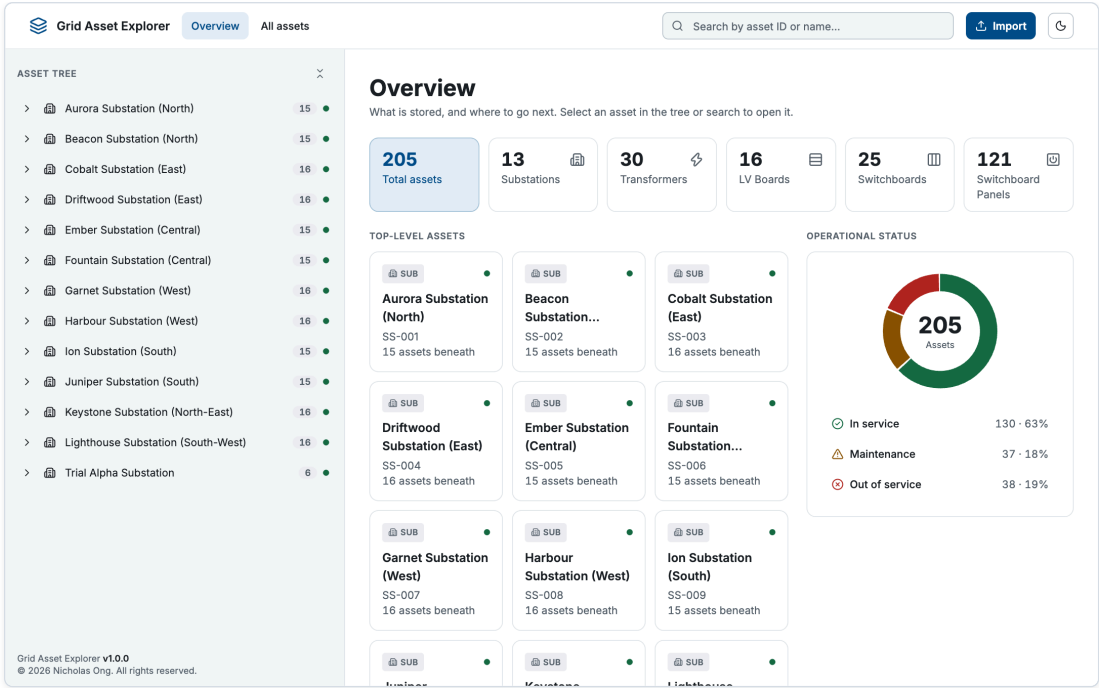


Figure 13.8 — Overview after importing the sample file: 205 assets (13 substations, 30 transformers, 16 LV boards, 25 switchboards, 121 switchboard panels).

13.5 Browsing the hierarchy

The asset tree

The tree on the left lists substations. Press the arrow beside a node to open it: its children load the first time you open it. The small grey number on a top-level node is the total number of assets beneath it (the per-type breakdown is on its details page, Fig. 13.9); the coloured dot is the operational status (green in service, amber maintenance, red out of service). The **double-chevron button** in the tree's header collapses everything.

Selecting an asset: the details area

Click any asset (in the tree, the Overview, a table, or a search result). The right-hand area shows its type and status, name and ID, voltage, rating, commissioned date, manufacturer, model, serial number and **parent** (a link). Below the fields, **Contents by type, at every level** has one card per type that can sit under it. The big number is how many of that type there are anywhere beneath the asset, however deep, and the line under it says how many of those are direct children: for a substation, its transformers, LV boards, switchboards and switchboard panels (a type with none shows 0; a switchboard panel, which cannot have anything beneath it, has no cards). Under **Immediate children**, a table lists the direct children. Click a heading (Name, Type, Status, Rating, Commissioned) to sort, and again to reverse. Empty values sort last.

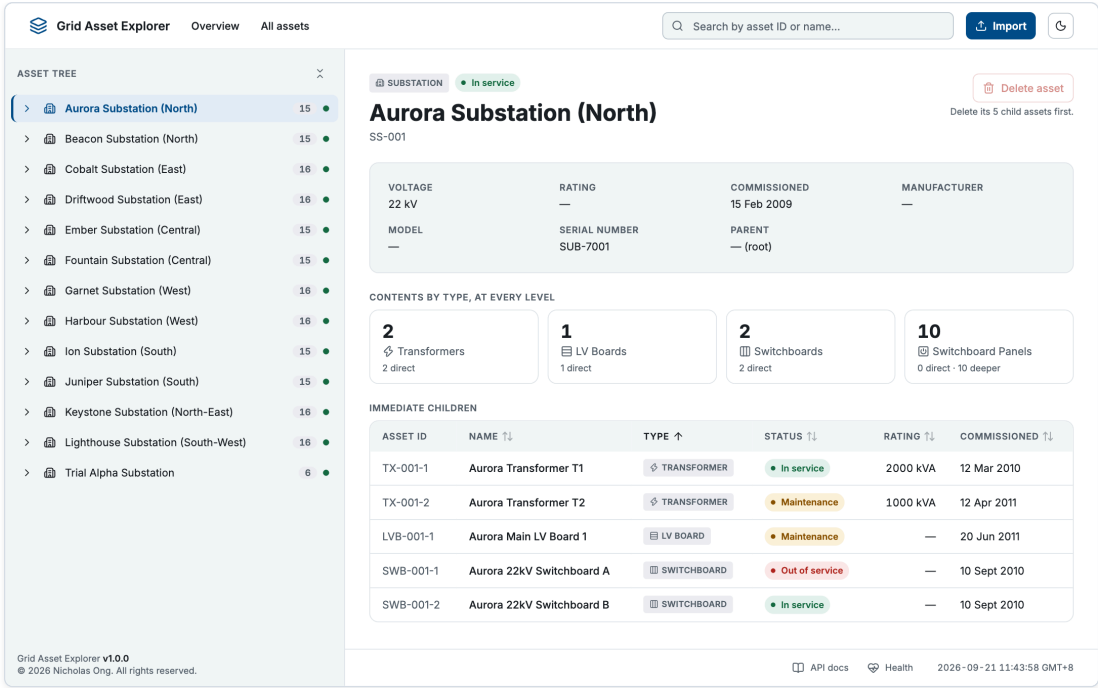


Figure 13.9 — A substation selected: its details; the contents by type at every level (2 transformers, 1 LV board, 2 switchboards, 10 switchboard panels, of which 0 are direct and 10 deeper); and its immediate children, sorted by type. The version and copyright are at the foot of the tree, and the docs and health links and the clock at the end of the page. **Delete asset** is dimmed with the note “Delete its 5 child assets first.”

Searching

Type in the top-bar search box; results appear after a short pause (300 ms), with an icon, name and ID. Use **↓** / **↑** to move, **Enter** to open, **Esc** to close. Up to 20 matches are shown, and the list says when there are more. Matching is partial and ignores case, on ID or name, with exact ID matches first.

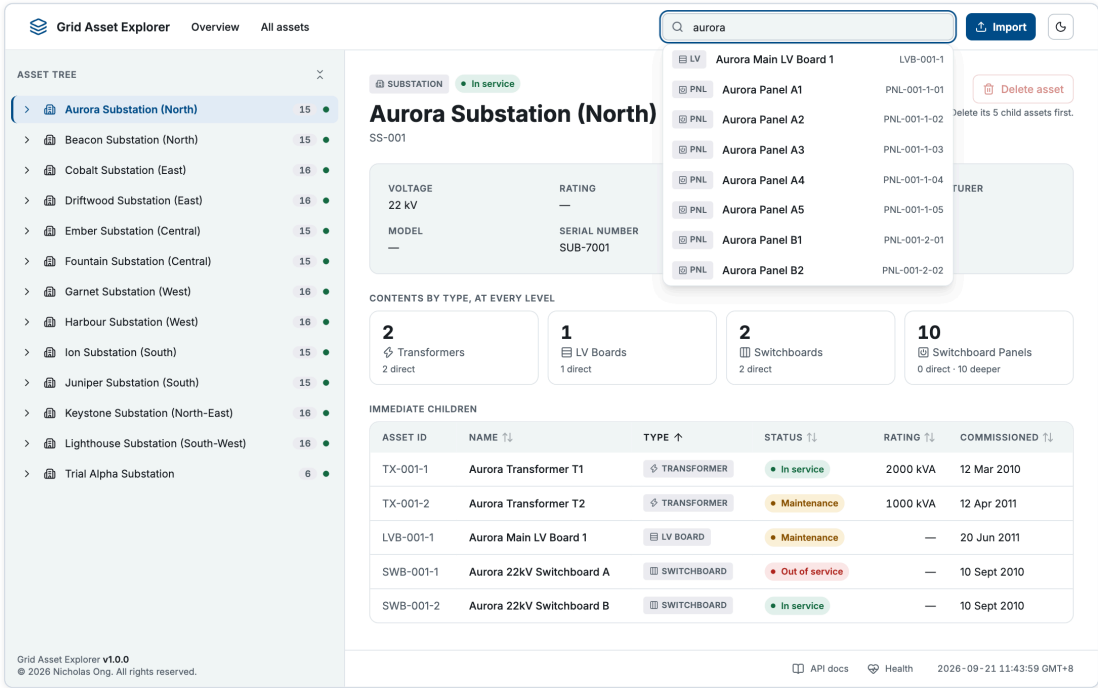


Figure 13.10 — Searching “aurora”.

Opening a result reveals it in the tree: the tree opens every ancestor and highlights the asset, so you always see where it sits (Fig. 13.11).

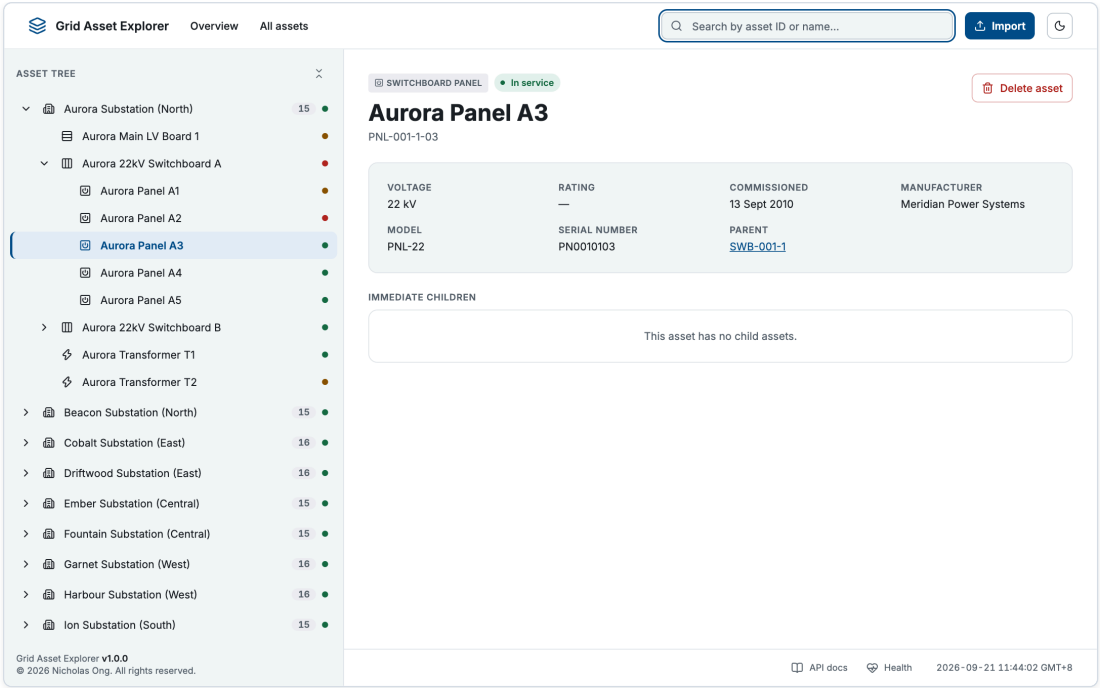


Figure 13.11 — After opening “Aurora Panel A3” from search: Substation → Switchboard A are expanded and the panel is highlighted. A panel has no children, so the child area says so.

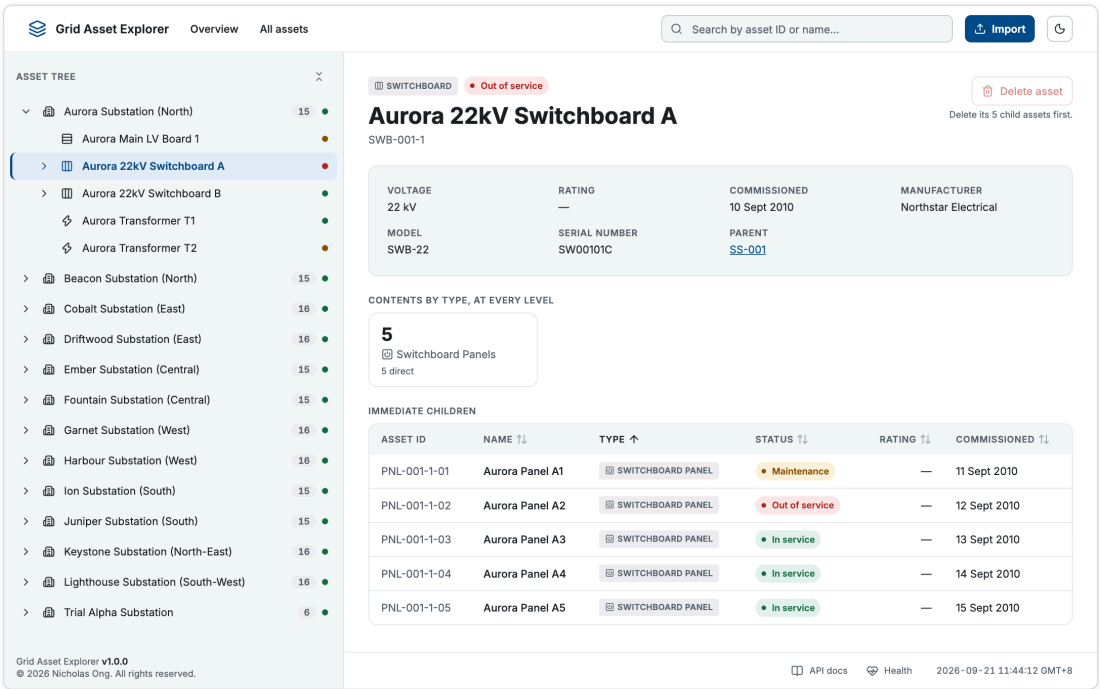


Figure 13.12 — A switchboard and its five switchboard panels: only the types that can sit under a switchboard are counted.

Asset-type icons

Each type has its own icon and short code, the same in the tree, the badges, the search results and the details.

Icon	Type	Code	Meaning
	Substation	SUB	A building.
	Transformer	TX	A lightning bolt.
	LV board	LV	Rows: the feeder breakers of a low-voltage board.
	Switchboard	SWB	Columns: a line-up of cubicles.
	Switchboard panel	PNL	A square with a power symbol: one switch unit.

The footer

The **version** (v1.0.0) and the copyright notice are at the very bottom of the tree on the left (at the bottom of the drawer on a phone; on the Import page, which has no tree, at the left of the footer). At the very end of each page, at the bottom right, are two links, **API docs** (the Swagger page, in a new tab) and **Health** (the health check), and a **clock** that shows the current date and time in your own time zone. The footer is not pinned to the screen: scroll to the end of the page to see it (on a short page it is already in view).

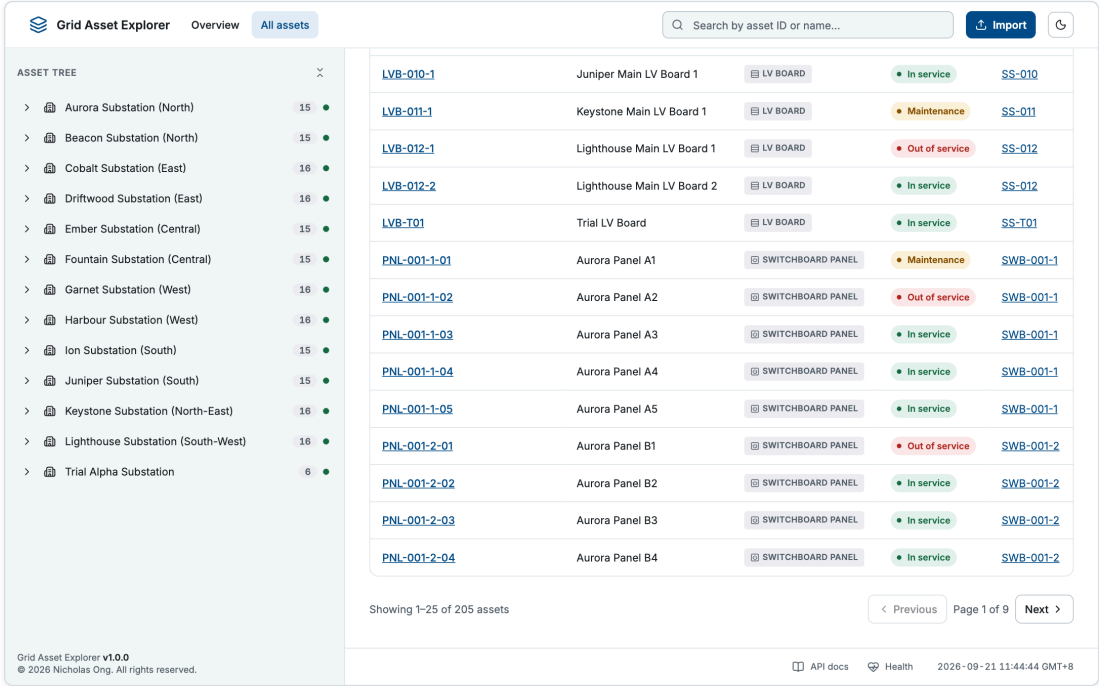


Figure 13.12b — The end of a long page (All assets): the version and copyright at the foot of the tree, and the API docs and Health links beside the clock at the bottom right.

13.6 The All assets list

A table of every stored asset. Type in **Filter by asset ID or name**, or use the **type** and **status** drop-downs. Click a column heading (Asset ID, Name, Type, Status, Parent) to sort; click again to reverse. The **server** sorts, so the order is correct across every page, not just the one on screen. Assets with no parent sort last when sorting by parent. Results are shown 25 per page. The filters and sort are kept in the address bar, so you can bookmark or share a view. IDs and parents are links.

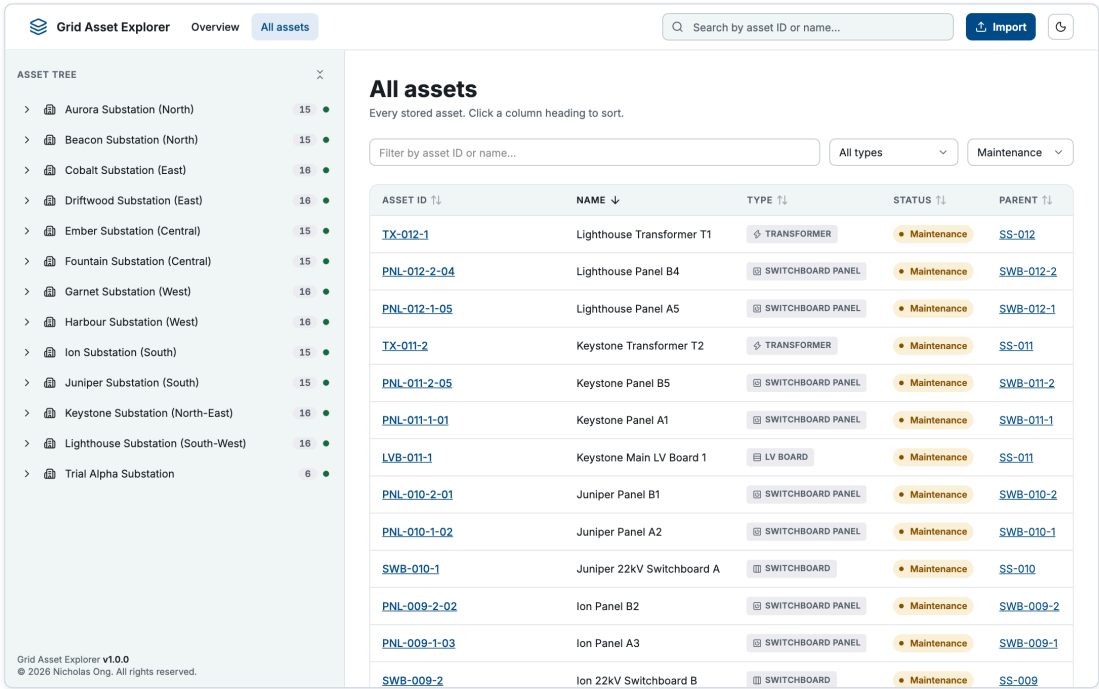


Figure 13.13 — All assets filtered to *Maintenance* and sorted by name, descending.

13.7 Deleting an asset

Deleting is for correcting mistakes and is only possible for an asset with **no children**. Open the asset and press **Delete asset**. A confirmation names the asset; choose **Delete asset** to confirm or **Cancel** (or **Esc**) to keep it. On success you are taken to its parent. To remove a whole branch, delete from the bottom up: panels, then switchboards, then the substation. There is no undo and no “delete everything”.

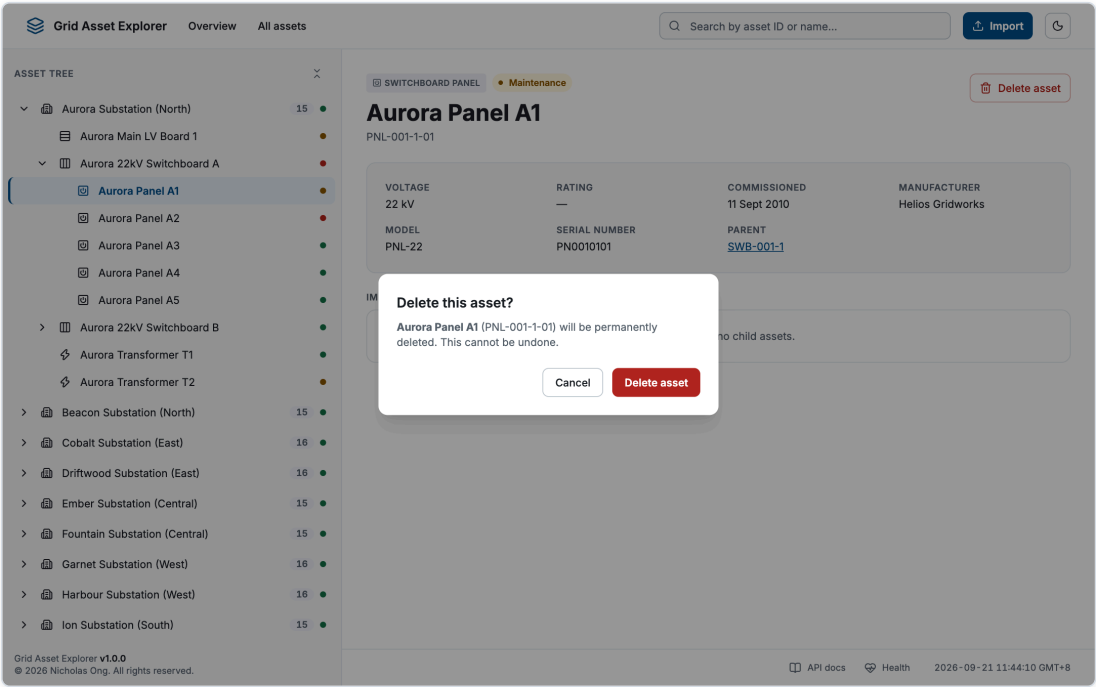


Figure 13.14 — The delete confirmation.

13.8 Dark mode, phones and states

Dark mode. The moon/sun button at the right of the top bar switches theme. The first visit follows your operating system; your choice is then remembered in this browser.

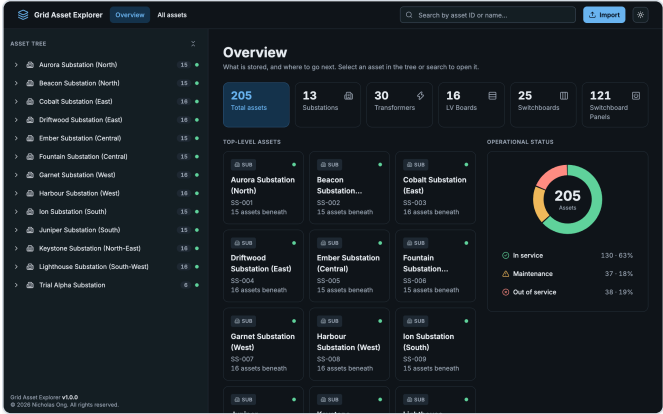


Figure 13.15a — Overview, dark

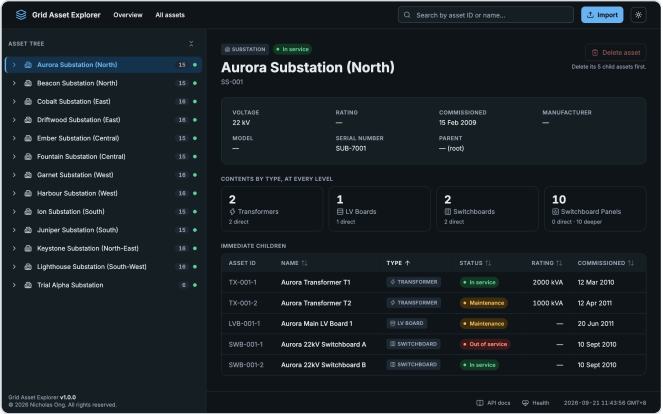


Figure 13.15b — Details, dark

Phones and small screens. Below tablet width the tree is not permanently visible: press the **Assets** button to slide it in as a drawer (it closes when you pick something). Wide tables scroll sideways with the ID column kept on one line. The Import cards stack.

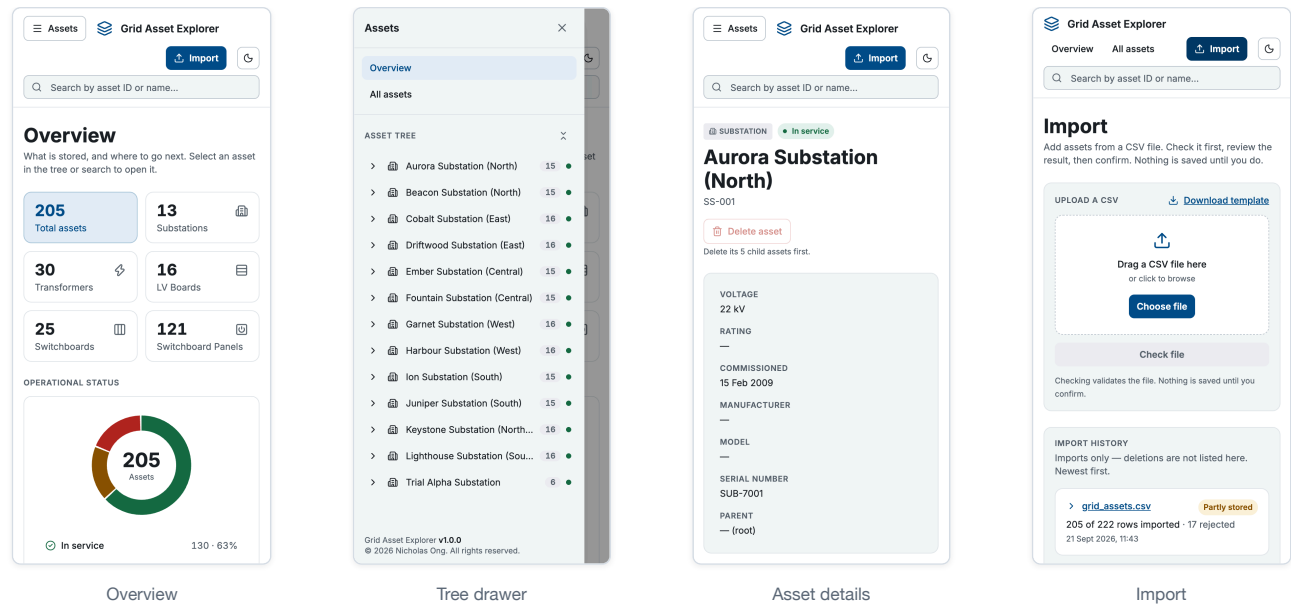


Figure 13.16 — The same pages on a 390 px wide phone.

Loading, empty and error states. Each panel shows its own state independently. While data loads you see grey placeholder bars; if a request fails the panel says what could not be loaded, shows the server's message, and offers **Try again**; a panel with nothing to show says so and suggests the next step. An unknown asset address shows a not-found panel.

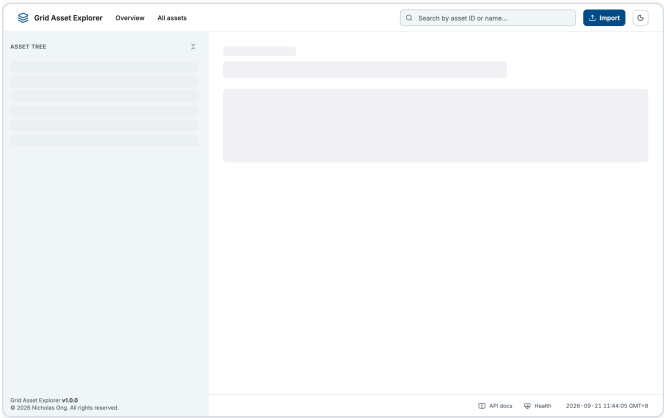


Figure 13.17a — Loading

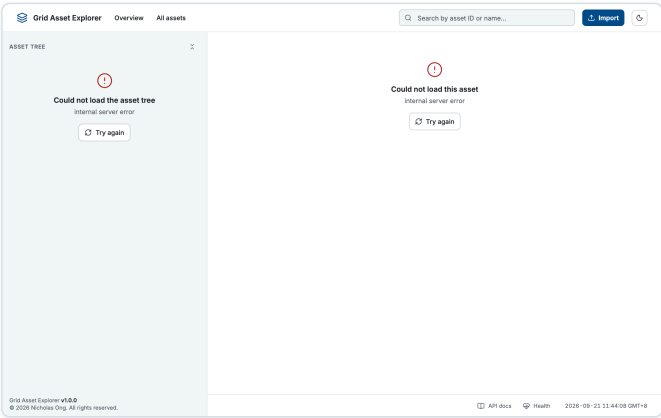


Figure 13.17b — Error with **Try again** (each panel separately)

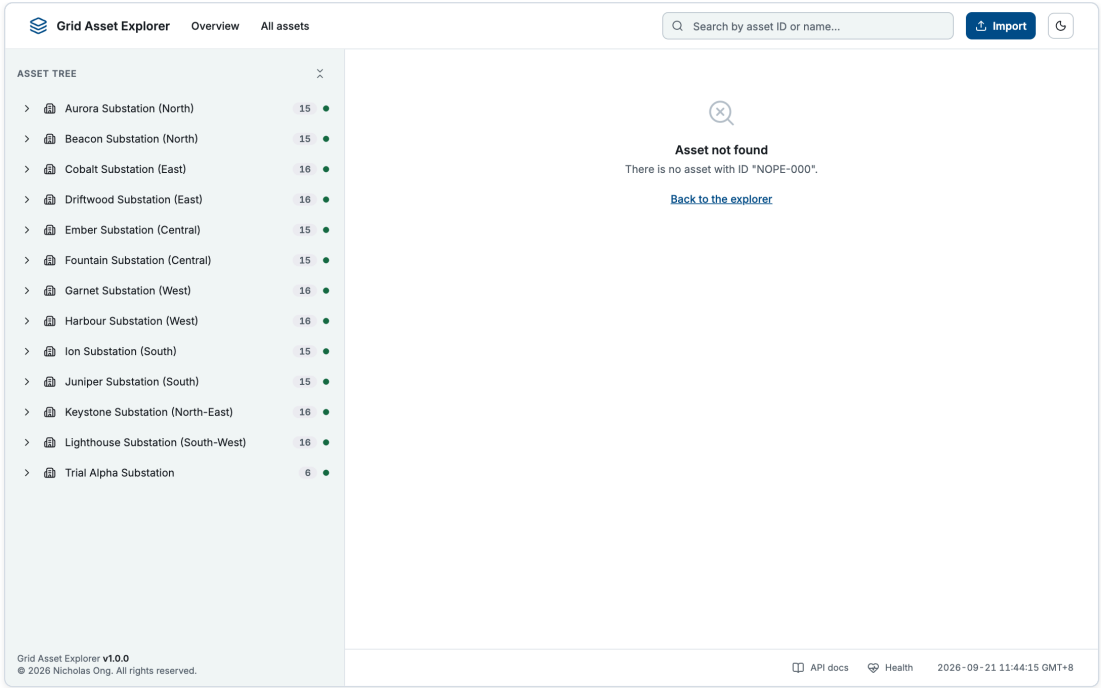


Figure 13.18 — An asset address that does not exist.

13.9 Keyboard reference

Where	Keys
Asset tree	<code>Tab</code> into the tree · <code>↑</code> <code>↓</code> move · <code>→</code> expand · <code>←</code> collapse · <code>Enter</code> select
Search	Type · <code>↓</code> <code>↑</code> choose · <code>Enter</code> open · <code>Esc</code> close
Dialogs (delete confirm, drawer)	Focus is held inside; <code>Esc</code> or Cancel closes
Anywhere	A visible focus ring always shows where you are; every control is reachable with <code>Tab</code>

13.10 Troubleshooting

What you see	Likely cause and what to do
Panels show “Could not load ...” with Try again	The backend is unreachable or failed (a 502 from the web server means the backend is not up yet). Wait a few seconds and press Try again ; if it persists ask your administrator to check <code>docker compose ps</code> and <code>/healthz</code> . Quote the <code>trace_id</code> if shown.
“header has only N column(s) ... check the file is comma-delimited”	The file is not a comma-separated CSV (for example, semicolons or tabs). Re-save it as CSV (comma delimited).
A message naming a missing column	The header row lacks one of the four required columns (<code>asset_id</code> , <code>asset_type</code> , <code>asset_name</code> , <code>operational_status</code>).
“request body too large”	The file exceeds the upload limit (10 MiB by default). Split it into several files, parents first.
File rejected as not a CSV	Only <code>.csv</code> files are accepted.
“file has a header but no data rows”	The file contains only the heading row.
A line number in the error (e.g. “line 42”)	A row has the wrong number of cells, usually an unquoted comma inside a value. Put the value in quotes.
Every row rejected as “already exists”	You re-uploaded a file that was already imported. Existing assets are never updated.
Many requests fail suddenly	Rate limit (100 requests per 10 s). Wait a few seconds.
Delete asset is dimmed	The asset has children. Delete them first.
My import review disappeared after refreshing the page	The chosen file is held in memory only. Choose the file again.

13.11 For administrators

- **Start:** `cp .env.example .env`, set `API_KEY` (`openssl rand -hex 32`), then `docker compose up --build`.
- **Health:** `http://localhost:8081/healthz` (or `:8080`) returns `{"status":"ok","db":"up"}`; it is also linked as **Health** in the footer.
- **API documentation:** the **API docs** link in the footer, or `http://localhost:8081/swagger/index.html` (Fig. 13.19). On port 8081 you can try requests straight away, because the web server adds the key. On the backend's own port, `http://localhost:8080/swagger/index.html`, press **Authorize** and enter the `API_KEY`; calls without the key get 401.
- **Start over:** `docker compose down -v` deletes the database volume (irreversible).
- **Logs:** `docker compose logs backend`. Every line and every error message carries the request's trace ID.
- **More commands** (update, stop, restart, tests, troubleshooting) are in chapter 11 and in `docs/DEPLOYMENT.md`. There is no CI/CD pipeline; the tests are run by hand.

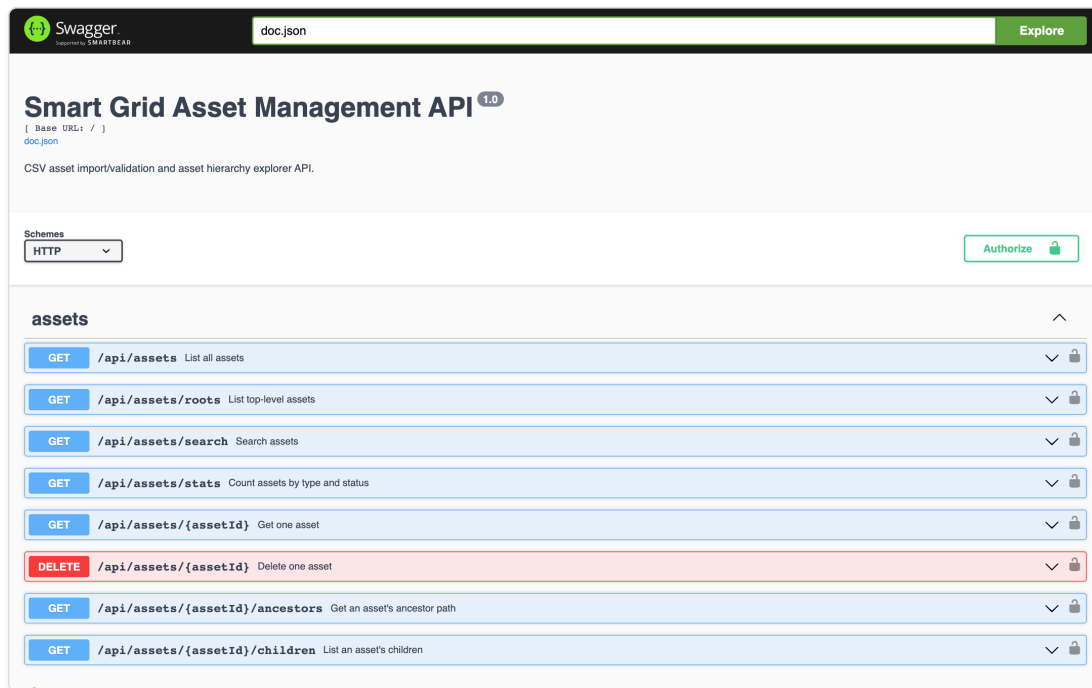


Figure 13.19 — Swagger UI, opened from the footer's API docs link (open, no key needed to read).

13.12 Frequently asked questions

Can I edit an asset? No. The system is import-first; to change an asset, delete it (leaves first) and import the corrected row.

Can I undo an import? Not with one click. Delete the imported assets from the bottom up, or ask an administrator to reset the data.

Why are some rows skipped but the rest imported? The system commits the valid rows and reports the rest, because one bad row in a large file should not block the others. You always see the outcome and can cancel before anything is saved.

Does the order of rows matter? No. A child may come before its parent in the file.

Who can see or change data? Anyone who can open the application can use every feature, including delete. There are no user accounts. Protect access to the application at the network level.

14. Known limitations and gaps

This chapter lists what was consciously left out or is a known trade-off, drawn from the repository's `docs/KNOWN_LIMITATIONS.md`. Every functional requirement from the brief (FR-1.x to FR-3.x) is met; the one gap found while preparing the first issue of this document (FR-3.4, the per-type counts beneath a substation) was closed in v1.1. Version 1.2 (21 September 2026) removed the never-run CI pipeline, replaced the LV board, switchboard and switchboard panel icons, added the footer, and dropped the sortable **Beneath** column from the children table (the contents-by-type cards remain and meet the requirement).

14.1 Limitations

Limitation	Detail
Shared key, not user accounts	Anyone who can use the UI can use every feature, including delete. The key protects the backend port from direct callers. Needs TLS beyond localhost.
Deletions not audited in the DB	Logged only; the import history cannot show them.
Skipped rows are not stored	Only line number, asset id and reason are kept; the original cell values are not. Existing assets are never updated (no upsert), so re-importing rejects everything already stored.
Application-layer hierarchy rules	A raw SQL insert could bypass the checks; nothing in the system does one.
Lookup snapshot	<code>GET /api/lookups</code> and the search <code>type</code> check read the snapshot without refreshing; a row inserted directly into a lookup table shows up after the next import or restart.
Ragged rows fail the whole file	Deliberate: an unquoted comma would silently shift columns. The message names the line.
Unbounded lists	The rejection list, <code>/roots</code> and <code>/children</code> are not paginated; the tree does not virtualize. Fine at hundreds of assets.
Precision	Numbers pass through <code>float64</code> (about 15 significant digits) before <code>NUMERIC</code> .
Ambiguous dates	Read day-first; a month-first supplier file with all days ≤ 12 would be misread without an error.
Rate limit is per process	100 requests / 10 s per (client, path).
Pending review lost on refresh	The chosen file is held in memory only.
Swagger is open	Docs are readable by anyone who can reach the backend port; calls still need the key.

14.2 Not done, and what would come next

- **No CI/CD, by design:** only a local build and run is needed, so nothing runs the tests automatically; run them by hand before a release (chapter 11). There is no registry, remote target or automated deploy.
- **No automated browser end-to-end or screen-reader tests.** The screenshots in this manual are a manual pass and could be scripted into a regression suite (the capture script used is in `docs/manual/capture-screenshots.mjs`).
- Real user accounts and a database audit trail for deletes; editing and upsert; persisting extra CSV columns as `JSONB`; pagination for the unbounded lists.

15. AI usage journal

This chapter condenses `docs/AI_JOURNAL.md` in the repository, which is the full record of how AI was used to build this project, reconstructed from the Claude Code session transcripts and the git history. It omits that file's session-by-session detailed log and the representative-prompt transcripts; see the file itself for those.

15.1 Tools, models and effort

Tool and model: Claude Code (VS Code extension), Claude Sonnet 5 in every recorded session. **Sub-agents:** 14 runs, all Claude Sonnet 5 — *Explore* agents did read-only research (repository state, the reference project, existing docs); *Plan* agents produced designs (schema changes, the CSV package, the frontend theme). Two Plan agents failed on a session rate limit at 16:40, and the design was done directly from the code instead. Spec-kit was not used.

Effort: about three hours hands-on inside a 3.6-hour window (14:40–18:16) on 2026-09-20, across eight sessions, 69 recorded human inputs (instructions, mid-run redirections and answers to the agent's questions) and 14 sub-agent runs. Eight sessions add up to about 4.5 hours of session time because up to four ran at once; sessions from S9 onward are not counted in these figures.

15.2 How the work was organised

- **Foundations first, in one session (S1).** The scaffold, documentation set, backend layering and shared packages were established before any feature work, so later sessions could stay small.
- **One concern per session, run in parallel.** Database design, logging, file processing and the frontend theme were four separate sessions started within 30 minutes of each other (S2–S5), each with its own plan.
- **Research, then decide, then build.** Sub-agents read the repository and the reference project; the agent then put concrete options to the owner as questions (with a recommendation); the owner chose; only then did it edit.
- **Reuse over invention.** For logging, database setup, middleware, the docs route and generated frontend types the agent was pointed at the reference project and asked for a simplified port, not a copy.
- **Every decision is written down:** `docs/ASSUMPTIONS.md` (choices), `docs/KNOWN_LIMITATIONS.md` (trade-offs), and the AI journal.

15.3 Session timeline

Session	Time	Workstream	Outcome
S1	14:40–15:49	Project initialisation and structure	Docker Compose stack, Go backend skeleton, Vite React + TypeScript frontend, DB migration, documentation set, layered backend, shared <code>base/</code> packages, middleware, CI skeleton
S2	15:39–16:03	Database design	Lookup tables, timestamps everywhere, client IP on imports, parent-type rules in the database, all-or-nothing import; class diagram added to the architecture document
S3	15:40–16:05	Structured logging	Trace-ID context, one-line structured format on <code>log/slog</code>
S4	15:59–16:16	Backend file processing	<code>base/csvx</code> package with line-accurate CSV parsing and 19 tests
S5	16:07–16:23	Frontend visual direction and licence	Palette tokens, system-font decision, proprietary licence file
S6	16:21–17:17	Step 1 wrap-up and the backend API	Scaffold committed; import pipeline, endpoints, Swagger, tests
S7	17:16	Docs route	<code>/docs</code> redirects to the Swagger UI
S8	17:14–18:16	Restructure, conventions, generated types, documentation	<code>service/</code> 19 to 8 files, gin + GORM, generated frontend types, docs sweep, the AI journal
S9	18:25–20:50	Frontend, real data, two-step import, API-key delete	Explorer UI, check → review → confirm import, overview, all-assets, activity, delete one, drawer
S10	21:00–22:14	Review refinements	One Import page, sorting, spacing scale, dark mode, status donut
S10 side sessions	21:49–22:05	One-off requests	Status panel, wider import history, a git question, a CI provider swap that was reverted, centred content
S11	22:11 onwards	Closure check and documentation alignment	Four gaps fixed, docs and the AI journal aligned
S12	2026-09-21	Local-only deployment, icons, footer, documentation refresh	CI pipeline removed, <code>docs/DEPLOYMENT.md</code> , three new type icons, footer with version and clock, docs revised

Session	Time	Workstream	Outcome
S13	2026-09-22	Documentation cleanup and AI journal chapter	GitLab references genericised throughout the docs and PDF, this chapter added, 44 pages

15.4 Suggestions rejected, corrected or revised

Who	What	Outcome
Owner	Asked for React; the architecture doc said TypeScript	The agent asked; TypeScript kept
Agent	Port only the core of the reference project's logger (its Kafka, Redis and gRPC sinks add unused dependencies)	Accepted
Agent	Replace the hand-rolled logger with <code>log/slog</code>	Accepted; a small custom handler added to control field order
Owner, then agent	A "sync DB" migration option was requested, then questioned	The agent recommended removal as speculative; removed
Agent	Moving all Markdown into <code>docs/</code> would stop <code>CLAUDE.md</code> auto-loading	Kept at the root; duplicate <code>AGENTS.md</code> dropped
Agent	The grading file would commit nothing under strict all-or-nothing	Kept all-or-nothing; trade-off documented
Agent	"Microservices" contradicts the single-backend design	Reinterpreted as goroutines in one backend
Agent	Would not guess the parent-type pairs	Asked the owner for the exact pairs
Spec, as written	Detect cycles only among rows passing earlier checks	Found by a test to make "cycle detected" unreachable; detection now runs over every in-file parent link
Agent	Send a large id list with <code>IN ?</code>	Would exceed Postgres's 65,535-parameter limit; single array parameter plus a 70,000-id test
Agent	Keep pass 1 and pass 2 in separate files	Overridden: one <code>service/import.go</code>
Owner	<code>gomock</code> for repository tests	Agent declined for that layer; offered for handler and service tests
Brief	React 18, Tailwind config file, MSW mocks	Repo already had React 19 and Tailwind 4, and the backend is ready: kept React 19 and Tailwind 4, no MSW; recorded as assumptions
Concept PDF	Import result shows "214 imported / 8 rejected" as committed	The API is all-or-nothing, so the UI leads with committed yes/no and shows imported as 0 when nothing was stored
Brief	Stop and show the skeleton before building the panels	The same message asked for a complete draft, so approving the plan was treated as the checkpoint; the agent said so in the plan
Owner, then agent	Agent asked "partial or all-or-nothing?" for the real file, which has ~17 bad rows	Replaced with check → review → confirm (validate first, then confirm, re-validating to avoid races); the agent designed the fingerprint/412/409 flow
Agent	Considered staging the uploaded file server-side between preview and confirm	Rejected by the agent itself: the browser keeps the file and re-uploads it, so there is no staging table, expiry or cleanup, and the backend never trusts a client-sent outcome
Owner	Import activity page	Agreed, then narrowed: label it "imports only" (deletions are not in the import tables), no new migration
Agent	Tree rows use <code>aria-owns</code> for their children	Tests exposed that an expanded row's accessible name then included its whole subtree; fixed with <code>aria-labelledby</code> on each row
Owner	Model-to-DTO helpers on the model	Agent flagged that <code>model</code> then imports <code>dto</code> ; built as asked and documented
Agent, then owner	Standard-library router recommended and kept at 15:33	Later replaced by gin at the owner's request, with the deviation recorded
Owner	Expand all button beside Collapse all in the tree	Built, then removed at the owner's request: collapse only
Owner, then agent	"Sorting" meant the All assets table, not the children list	The agent added server-side <code>sort</code> and <code>dir</code> , because sorting only the 25 rows on screen would mislead
Owner	Swap the CI provider	The agent did it; reverted: kept the original pipeline
Owner (S12)	Remove the CI pipeline altogether; a local build and run is enough	Done, with the assumption recorded and <code>DEPLOYMENT.md</code> written; the earlier decision to keep it was superseded

Who	What	Outcome
Agent, then owner (S12)	Three icon sets proposed for the LV board, switchboard and panel	Rows3 / Columns3 / SquarePower chosen
Agent, then owner (S12)	Footer clock: browser time zone, UTC or build time	The live browser-zone clock chosen
Agent, then owner (S12)	Footer as a bar fixed to the bottom of the screen	Changed to the reference's layout: copyright at the foot of the sidebar, links and clock at the end of the page
Owner (S1), then owner (S11)	Swagger docs behind the API key	Removed in S11: with the endpoints open at the time, the gate protected nothing
Owner	Only DELETE needs the key	Changed in S11: every <code>/api</code> endpoint needs it; only <code>/healthz</code> and the docs are open

15.5 Known gaps and next steps

Full list in `docs/KNOWN_LIMITATIONS.md`. The gaps that matter:

- **The whole `/api` is guarded by one shared API key** (no user accounts) that nginx adds itself, so the UI's users can still do everything, including delete; deletions are only in the server log, not on the Import activity page.
- **The frontend has only had a light browser check.** S10 and the closure check (S11) looked at the main pages in headless Chrome (both themes, desktop and 390 px, against the real data), but not at focus rings or drag-and-drop. The delete dialog, import review and report screens, loading and error states and the mobile drawer were captured on 2026-09-21 for this manual. There is no end-to-end test and no screen-reader check.
- **A pending import review is lost on refresh** (the chosen file is held in memory); past imports are on the Import activity page.
- **No CI/CD, by design.** Only a local build and run is needed, so there is no pipeline and no remote target; the tests are run by hand (`docs/DEPLOYMENT.md`).
- **Bulk-insert speed is unmeasured** since the move to GORM.

Appendix A. CSV template and sample data

The downloadable template (`GET /api/imports/template`) is generated from the importer's own column table, so it cannot drift from what the importer accepts. Its four example rows form a valid mini-hierarchy:

```
asset_id,asset_type,asset_name,operational_status,parent_asset_id,voltage_kv,rating_kva,manufacturer,model,serial_number,commissioned_date
EXAMPLE-SUB-001,SUBSTATION,Example Substation,IN_SERVICE,,22,,Example Manufacturer,SS-100,EX-SS-0001,2020-01-31
EXAMPLE-TX-001-1,TRANSFORMER,Example Transformer 1,IN_SERVICE,EXAMPLE-SUB-001,22,1000,Example Manufacturer,EX-TX-1000,EX-TX-0001,2020-06-15
EXAMPLE-SWB-001-1,SWITCHBOARD,Example Switchboard A,MAINTENANCE,EXAMPLE-SUB-001,22,,,,,2021-03-01
EXAMPLE-PNL-001-1-1,SWITCHBOARD_PANEL,Example Panel A1,OUT_OF_SERVICE,EXAMPLE-SWB-001-1,,,,,
```

The supplied sample, `backend/testdata/grid_assets.csv`, uses the column order `asset_id`, `parent_asset_id`, `asset_type`, `asset_name`, `operational_status`, `voltage_kv`, `rating_kva`, `manufacturer`, `model`, `serial_number`, `commissioned_date` (note `parent_asset_id` second) and day-first two-digit-year dates such as `15/2/09`. It imports cleanly with the column order differing from the template, which demonstrates that columns are matched by name.

Appendix B. Glossary

Term	Meaning
Asset	A piece of grid equipment identified by a unique <code>asset_id</code> .
Substation	Root asset type; has no parent.
Transformer, LV board, switchboard	Assets that belong to a substation.
Switchboard panel	An asset that belongs to a switchboard.
Root type	A type with no permitted parent (read from <code>asset_type_parent_rules</code> , not hard-coded).
Preview / Check	Validating a file without storing anything.
Fingerprint	SHA-256 identifying a file <i>and</i> its validation outcome; lets confirm detect that things changed since the preview.
Cascading rejection	A valid row rejected because an ancestor was rejected.
Trace ID	8-hex identifier of one request, present in the response header, error body and every log line.
CTE	Common table expression; the recursive form computes subtree counts, descendant counts and ancestor paths in one query.